BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION Artificial Intelligence

DIPLOMA THESIS

Real-Time Intrusion Detection for Web Applications Using Deep Learning

Supervisor
Alexandra–Ioana Albu

Author
Rares Marta

2026

ABSTRACT

This thesis designs, implements, and evaluates a real-time intrusion detection system (IDS) for IoT networks using deep learning on the CICIoT2023 benchmark dataset. The work is situated at the intersection of artificial intelligence and cybersecurity and delivers a complete, reproducible pipeline from raw packet captures to live per-flow classification.

The system is built around a two-hidden-layer Multi-Layer Perceptron (MLP) trained at binary and multi-class (8 attack families) granularities on a temporally ordered train/validation/test split, ensuring that evaluation reflects deployment conditions rather than optimistic random-sampling results. The MLP is compared against Random Forest and XGBoost baselines under identical conditions. On the 8-class temporal test set, all three models exceed the 0.85 weighted-F1 target, with XGBoost leading at 0.929 and the MLP at approximately 0.901. Feature analysis identifies TCP protocol type, header length, and TCP flag counts as the most discriminative features, and reveals that 14 of the 39 publicly available CICIoT2023 features carry near-zero empirical variance and are pruned prior to training.

The practical component includes a web demonstration — NEURAL IDS — comprising a FastAPI backend and a React dashboard that accepts pre-extracted feature CSVs or raw packet captures, classifies network flows in under 10 milliseconds, and displays confidence scores and per-class probability breakdowns. The demonstration confirms that the system correctly identifies a synthetic SYN-flood capture as an attack (with 100% binary confidence) and classifies benign browsing traffic as Benign, validating the core F1 functionality required by the project specification.

Contents

Chapter 1

Introduction

Intrusion Detection Systems (IDS) are a core defensive layer in modern networks, monitoring traffic to detect malicious behaviors such as denial-of-service attacks, reconnaissance scans, brute-force attempts, and data exfiltration. As the number of connected devices continues to grow — particularly in the Internet of Things (IoT) — the volume and diversity of network traffic have increased dramatically, exposing the limitations of traditional signature-based approaches that rely on manually maintained rule sets.

Deep learning offers a promising alternative by learning complex, non-linear representations directly from network traffic data, enabling models to detect both known and previously unseen attack patterns. However, a gap remains between the strong benchmark results reported in the research literature and the practical deployment of such models in real-time environments.

This thesis addresses that gap by designing, implementing, and evaluating a deep learning-based intrusion detection system capable of operating in real time. The work focuses on the CICIoT2023 dataset, a large-scale IoT intrusion detection benchmark containing over 46 million labeled network flow records across 33 attack types. A Multi-Layer Perceptron (MLP) is developed as the primary supervised classifier, evaluated against tree-based baselines (Random Forest, XGBoost) on the same train/validation/test splits to verify that the deep model is the best choice for this tabular task. Beyond offline evaluation, the thesis includes a demonstration environment in which scripted attacks are launched against a controlled web application, and the trained model classifies the resulting traffic flows in near real time.

The remainder of this thesis is organized as follows:

- **Chapter 2** presents the theoretical background, covering intrusion detection systems, deep learning for traffic classification, and the CICIoT2023 dataset.
- **Chapter 3** states the application requirements and specification, defining the problem to be solved, the actors involved, and the functional and non-functional

constraints the system must satisfy.

- **Chapter 4** describes the design, implementation, and testing of the application, including the data pipeline, model architecture, training procedure, the live-demo environment, and the verification of the core functionality.
- **Chapter 5** reports experimental results and discusses the findings.
- **Chapter 6** concludes the thesis and outlines directions for future work.

Chapter 2

Theoretical Background

This chapter presents the theoretical foundations underlying the work developed in this thesis. It begins with an overview of intrusion detection systems and their role in network security, then introduces deep learning as a tool for traffic classification, with a focus on multi-layer perceptrons. Finally, it describes the CICIoT2023 dataset, which serves as the training and evaluation basis for the models developed in subsequent chapters.

2.1 Intrusion Detection Systems

An Intrusion Detection System (IDS) is a security mechanism that monitors network traffic or host activity to identify malicious behavior. As networks have grown in scale and complexity, traditional perimeter defenses such as firewalls have proven insufficient on their own: attackers routinely bypass static rules and exploit new vulnerabilities, while modern networks exhibit high throughput, heterogeneity, and continual change [9]. IDS complement these defenses by analyzing traffic patterns and flagging suspicious activity.

IDS can be broadly categorized into two paradigms based on their detection methodology: signature-based and anomaly-based systems.

2.1.1 Signature-Based Detection

Signature-based IDS, exemplified by tools such as Snort and Suricata, operate by scanning network traffic for predefined patterns of known malicious activity [12]. These signatures can describe byte sequences, header anomalies, or protocol violations. Signature-based systems are highly effective at detecting known attacks with high precision, and they produce few false positives when the signature database is well-maintained.

However, their static nature makes them fundamentally limited against novel threats. Zero-day exploits, polymorphic attacks, and previously unseen attack variants will not match any existing signature and will therefore pass undetected. Additionally, signature databases require continuous manual updates to remain effective, which introduces an inherent lag between the emergence of a new attack and the availability of its corresponding signature.

2.1.2 Anomaly-Based Detection

Anomaly-based IDS take a different approach: instead of matching against known attack patterns, they learn a model of normal network behavior and flag deviations from this baseline as suspicious. This paradigm has the significant advantage of being able to detect previously unseen attacks, since any traffic that deviates sufficiently from the learned normal profile will trigger an alert.

The trade-off is a higher false positive rate. Unusual but legitimate traffic — such as a sudden spike in usage during a product launch, or an uncommon protocol used by a newly deployed application — can be incorrectly flagged as malicious. The sensitivity of anomaly-based systems depends heavily on the quality of the normal behavior model and on the threshold chosen to separate normal from anomalous activity.

Figure 2.1 illustrates the typical architecture of a real-time deep learning-based IDS, showing the end-to-end pipeline from raw packet capture to alert generation.

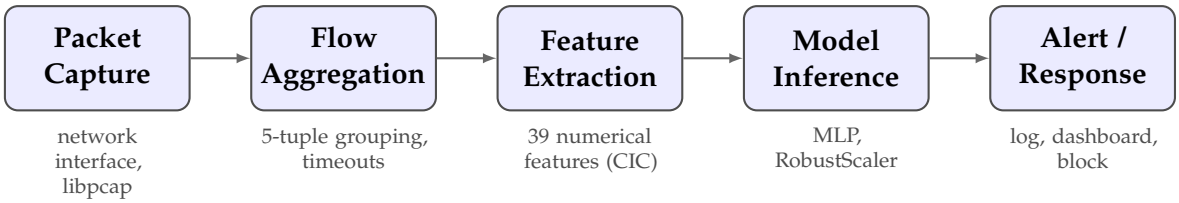

Figure 2.1: End-to-end architecture of a real-time deep learning-based intrusion detection system.

From a machine learning perspective, intrusion detection maps naturally to classification. In its simplest form, the task is binary: distinguish benign traffic from malicious flows. In practice, multi-class setups are common, with systems trained to recognize distinct categories of attacks such as DDoS, denial-of-service, reconnaissance, or botnet activity. The input is typically tabular flow data, where each flow is described by numerical features such as packet counts, byte rates, and TCP flag statistics [13].

2.1.3 Key Challenges

Deep learning models for IDS must operate under several challenges that frequently undermine the transfer from benchmark performance to real-world deployment:

- **Class imbalance**: In most network environments, benign traffic vastly outnumbers attack traffic. Benchmark datasets reflect this — for example, in CICIoT2023, DDoS attacks account for approximately 73% of all records, while web-based attacks represent less than 0.1% [11]. This imbalance can cause models to favor the majority class, making accuracy a misleading metric and requiring techniques such as weighted loss functions or stratified sampling.
- **Concept drift**: Network traffic distributions change over time due to evolving user behavior, new applications, updated protocols, and shifting attacker strategies. A model trained on a static dataset may degrade in production as the traffic it encounters diverges from its training distribution [3].
- **Dataset realism**: Most benchmark datasets are generated in controlled testbed environments rather than captured from production networks. While testbeds allow for precise labeling of attacks, they may not fully capture the diversity and complexity of real-world traffic, including encrypted communications, multi-stage attacks, and the noise inherent in large-scale networks [9].
- **Deployment constraints**: A practical IDS must meet latency and throughput requirements, integrate with existing security infrastructure, and remain maintainable over time. These operational aspects are frequently underreported in the research literature, which tends to focus on classification accuracy on static benchmarks [4].

2.2 Deep Learning for Network Intrusion Detection

Traditional machine learning approaches to IDS — such as Support Vector Machines, Decision Trees, and Random Forests — depend on manually engineered features and often struggle with high-dimensional, imbalanced, and non-stationary data. Deep learning offers an alternative by learning hierarchical representations directly from raw or lightly processed inputs. In the IDS domain, the literature reports a wide range of deep learning architectures, including deep feed-forward networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs) with Long Short-Term Memory (LSTM) units, autoencoders, and more recently transformer-based models with self-attention [9].

This section focuses on the multi-layer perceptron (MLP), which serves as the primary model in this thesis, and briefly surveys other architectures for context.

2.2.1 Multi-Layer Perceptrons

A Multi-Layer Perceptron (MLP), also referred to as a fully connected feedforward neural network or deep neural network (DNN), is the foundational deep learning architecture for supervised classification. An MLP consists of an input layer, one or more hidden layers, and an output layer, where each layer is fully connected to the next.

Formally, given an input feature vector $\mathbf{x} \in \mathbb{R}^d$ representing a network flow described by d numerical features, each hidden layer l computes:

$$\mathbf{h}_l = \sigma(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l) \tag{2.1}$$

where $\mathbf{W}_l$ and $\mathbf{b}_l$ are the learnable weight matrix and bias vector of layer l, and σ denotes a non-linear activation function, typically the Rectified Linear Unit (ReLU), defined as $\text{ReLU}(x) = \max(0, x)$. The input to the first hidden layer is the feature vector itself: $\mathbf{h}_0 = \mathbf{x}$.

For binary classification (benign vs. attack), the output layer applies a sigmoid activation:

$$\hat{y} = \sigma_{\text{sig}}(\mathbf{w}^T \mathbf{h}_L + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{h}_L + b)}} \tag{2.2}$$

where $\mathbf{h}_L$ is the output of the last hidden layer and $\hat{y} \in (0, 1)$ represents the predicted probability that the input flow is an attack. The model is trained by minimizing the binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{2.3}$$

using backpropagation and an optimizer such as Adam [14].

MLPs are particularly well suited for flow-level intrusion detection for several reasons:

- **Native fit for tabular data**: Network flow features form fixed-length numerical vectors — exactly the input format MLPs are designed for. Unlike CNNs (which assume spatial structure) or RNNs (which assume sequential structure), MLPs make no structural assumptions about the input, which is appropriate when each sample is an independent flow record with heterogeneous features [7].
- **Fast inference**: MLP inference consists of a sequence of matrix multiplications, which are highly optimized on modern hardware. This makes MLPs particularly suitable for real-time IDS, where low latency is critical. A small MLP can process tens of thousands of flows per second on a single CPU core [14].

- **Competitive performance on tabular tasks**: Recent research has shown that well-regularized MLPs are competitive with both tree-based models and more complex neural architectures on tabular classification tasks. Kadra et al. demonstrated that an MLP with a carefully tuned combination of regularization techniques (dropout, weight decay, batch normalization) outperformed both XGBoost and specialized architectures such as TabNet on nearly half of the 40 datasets evaluated [7].

- **Strong baseline**: In IDS research, MLPs consistently achieve 95–99% accuracy on standard benchmarks, making them an honest and competitive baseline against which to measure more complex architectures [14, 5].

Key training considerations for MLP-based IDS include:

- **Class imbalance handling**: Weighted loss functions assign higher weight to minority (attack) classes in the cross-entropy loss. Alternatively, oversampling techniques such as SMOTE can be used to generate synthetic minority samples, or focal loss can be applied to down-weight well-classified examples and focus training on hard cases.

- **Regularization**: Dropout (randomly zeroing neuron outputs during training with a probability typically between 0.2 and 0.5) prevents co-adaptation of neurons. Weight decay (L2 regularization) penalizes large weights. Early stopping monitors validation loss and halts training when it begins to increase.

- **Feature scaling**: Network flow features have vastly different scales (e.g., packet counts vs. flow durations in microseconds). Standard normalization — fitting a scaler on the training set and applying it to validation and test sets — is essential for stable gradient-based optimization.

2.2.2 Other Deep Learning Architectures for IDS

Beyond MLPs, several other deep learning architectures have been applied to intrusion detection:

- **Convolutional Neural Networks (CNNs)**: CNNs exploit local correlations by applying convolutions to reshaped tabular features, treating them as one-dimensional or two-dimensional grids. Convolutions can extract local patterns that correspond to protocol-specific or timing-related signatures of attacks [15].

- **Recurrent Neural Networks (RNNs/LSTMs)**: LSTM-based IDS capture temporal dependencies in sequential traffic data, which is particularly useful in

industrial or cyber-physical system settings where context over time is critical [8].

- **Autoencoders**: Autoencoders learn to reconstruct their inputs through a bottleneck latent representation. When trained only on benign traffic, the reconstruction error serves as an anomaly score: flows with unusually high error are flagged as attacks. This unsupervised approach does not require labeled attack data, but depends heavily on threshold selection and training data quality [10].

- **Transformers**: Transformers rely on self-attention to model global relationships among features. In tabular IDS adaptations, each feature can be treated as a token and embedded into a vector space. The key advantage is the ability to model global feature interactions rather than relying on fixed local receptive fields. However, transformer models are typically supervised, requiring labeled attack data, and can be compute-intensive [16].

Table 2.1 summarizes the key differences between supervised and unsupervised deep learning paradigms for IDS.

Aspect	**Unsupervised / Hybrid**	**Supervised**
Label requirement	Low–Medium	High
Zero-day sensitivity	Higher potential	Typically lower
Calibration effort	High (thresholding)	Medium
Benchmark performance	Medium–High	High
Inference speed	Fast	Varies

Table 2.1: Qualitative comparison of unsupervised/hybrid and supervised DL-IDS paradigms.

2.3 The CICIoT2023 Dataset

The CICIoT2023 dataset, published by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick, is a large-scale benchmark dataset for IoT intrusion detection research [11]. It was selected for this thesis based on several criteria: recency (2023), scale (over 46 million records), diversity of attack types (33 distinct attacks across 7 categories), and growing adoption in the research community.

2.3.1 Testbed and Data Collection

The dataset was generated in a physical laboratory environment simulating a realistic smart home network. The testbed consists of **105 real IoT devices**, including smart cameras, speakers, plugs, lights, sensors, and home automation controllers from various manufacturers. Of these, 67 devices are directly involved in attack scenarios as either attackers or victims, while 38 Zigbee/Z-Wave devices are connected through 5 hubs.

The attack infrastructure uses 7 Raspberry Pi 4 devices configured as attackers. For distributed attacks (DDoS), multiple Raspberry Pis coordinate via an SSH-based master-client configuration. Network traffic is passively captured using a Gigamon network tap and two monitoring stations running Wireshark, producing approximately 548 GB of raw PCAP data.

Feature extraction is performed using **DPKT**, a Python packet parsing library. Unlike CICFlowMeter (used in earlier CIC datasets such as CICIDS2017), DPKT operates at the packet level with a fixed-size packet window, computing features over a sequence of consecutive packets between two hosts rather than per-flow. This approach offers finer granularity and is more suitable for real-time detection, as features can be computed as soon as enough packets in the window are observed, without waiting for a flow to terminate [11].

2.3.2 Attack Taxonomy

The dataset contains **33 distinct attack types** organized into 7 categories, plus benign traffic, for a total of 34 class labels. Table 2.2 summarizes the attack categories, the number of attacks in each, and the tools used for generation.

2.3.3 Feature Set

The original CICIoT2023 paper describes a 46-feature representation produced by the DPKT-based extractor used during dataset construction. The features can be grouped as follows:

- **Flow timing** (3 features): flow duration, time-to-live (TTL), and inter-arrival time with the prior packet.
- **Rate and throughput** (3 features): overall packet transmission rate, source rate, and destination rate.
- **Header and protocol** (2 features): header length and protocol type (IP, UDP, TCP, IGMP, ICMP).

Category	Count	Examples	Tools
DDoS	13	UDP Flood, SYN Flood, SlowLoris, HTTP Flood	hping3, golang-httpflood
DoS	4	TCP Flood, UDP Flood, SYN Flood, HTTP Flood	hping3, udp-flood
Reconnaissance	5	PingSweep, PortScan, OSScan, VulnerabilityScan	nmap, fping, vulscan
Web-Based	6	SQL Injection, XSS, Command Injection, Backdoor	DVWA, Remot3d, BeEF
Brute Force	1	DictionaryBruteForce	nmap, hydra
Spoofing	2	ARP Spoofing, DNS Spoofing	ettercap
Mirai	3	GREIP Flood, GREeth Flood, UDP-Plain	Adapted Mirai source

Table 2.2: CICIoT2023 attack taxonomy: 7 categories with 33 attack types.

- **TCP flag features** (7 features): binary indicators for FIN, SYN, RST, PSH, ACK, ECE, and CWR flags.
- **Packet count features** (5 features): counts of ACK, SYN, FIN, URG, and RST packets.
- **Protocol indicator features** (14 binary features): binary indicators for HTTP, HTTPS, DNS, Telnet, SMTP, SSH, IRC, TCP, UDP, DHCP, ARP, ICMP, IPv, and LLC.
- **Packet length statistics** (6 features): total sum, minimum, maximum, average, standard deviation, and individual packet length.
- **Aggregate statistical features** (6 features): packet count in flow, magnitude, radius, covariance, variance, and weight — derived from combined incoming and outgoing packet statistics.

At the time of writing, the publicly available CSV release distributed by the Canadian Institute for Cybersecurity contains a **39-feature subset** of the 46 features described in the paper. Specifically, the public CSV omits the source and destination rate features, the URG packet count, and four of the six aggregate statistics (*Magnitude*, *Radius*, *Covariance*, and *Weight*). The 39-feature version is the basis for all experiments and the live demonstration in this thesis. This is acknowledged as a deliberate scope

decision: results in this work are not numerically directly comparable to papers that report on the full 46-feature variant, and that gap is part of the methodological story rather than an oversight.

2.3.4 Dataset Statistics and Class Distribution

The full dataset contains approximately **46.7 million records** distributed across 169 CSV files, totaling approximately 13 GB uncompressed. The class distribution is severely imbalanced:

- **Benign traffic**: approximately 1.1 million samples (2.4% of the dataset).
- **DDoS attacks**: approximately 73% of all records, with DDoS-ICMP_Flood alone accounting for over 7.2 million samples.
- **Web-based attacks**: fewer than 25,000 samples combined, with the rarest classes (XSS, Backdoor_Malware, CommandInjection) containing only around 100 samples each.

The most extreme class ratio is approximately 5,751:1 between DDoS-ICMP_Flood and the smallest attack class. This severe imbalance necessitates careful handling during model training, either through stratified sampling, class weighting, or a combination of both. Due to the dataset's size and computational constraints, most published experiments operate on stratified subsamples of 1–5 million records, preserving the original class distribution [11].

2.3.5 Comparison with Earlier Datasets

Table 2.3 compares CICIoT2023 with other commonly used IDS benchmark datasets.

Dataset	Year	Records	Features	Attack types	Domain
NSL-KDD	2009	150K	41 (custom)	4 classes	General
UNSW-NB15	2015	2.5M	49 (Argus+Bro)	9 classes	General
CICIDS2017	2017	2.8M	78 (CICFlowMeter)	14 attacks	Enterprise
CICIoT2023	2023	46.7M	39 (DPKT, CSV)	33 attacks	IoT smart home

Table 2.3: Comparison of commonly used IDS benchmark datasets.

CICIoT2023 offers several advantages over older benchmarks: it is significantly larger, contains more diverse and modern attack types (including IoT-specific attacks such as Mirai botnet variants), and uses real IoT hardware rather than simulated traffic generators. Its primary limitations include the lack of encrypted traffic analysis, the restriction to a smart home IoT context (excluding industrial or medical IoT), and the fact that attacks are executed individually rather than as multi-stage campaigns [11].

2.3.6 Known Limitations

Several limitations of the CICIoT2023 dataset should be acknowledged:

- **Simulated environment**: Although real IoT devices are used, the attacks are executed in a controlled laboratory setting. Real-world attackers combine techniques, adapt strategies, and conceal their actions — none of which are captured in a static dataset.
- **No encrypted traffic**: The features are extracted from packet headers and metadata. While HTTPS is flagged as a binary indicator, the dataset does not contain features derived from encrypted payload inspection. IoT traffic is increasingly encrypted, which limits the generalizability of models trained on this data.
- **Smart home scope**: The testbed represents a single domain. Industrial IoT, healthcare IoT, and other verticals are not represented, and models may not transfer to those environments.
- **Severe class imbalance**: As noted above, the most extreme ratio exceeds 5,000:1, requiring careful handling during both training and evaluation.
- **Feature documentation**: The exact formulas for some derived features (Magnitude, Radius, Covariance, Variance, Weight) are not fully documented with mathematical definitions in the original paper.

Chapter 3

Application Requirements and Specification

This chapter defines the application that constitutes the practical contribution of the thesis. The overall objective is twofold: to deliver a defensible, reproducible deep-learning pipeline trained on the CICIoT2023 dataset, and to expose the resulting model through a live demonstration in which the user can submit captured network traffic and observe per-flow intrusion-detection verdicts. The chapter starts from this objective, identifies the actors and use cases, lists the functional and non-functional requirements that shape the design, and concludes with the system-level specification that constrains the implementation in Chapter 4.

3.1 Project Goal and Scope

The deliverable is a real-time intrusion detection system (RT-IDS) for IoT networks built on top of CICIoT2023, structured as two cooperating subsystems:

- **Training subsystem.** A reproducible offline pipeline that ingests the raw CICIoT2023 captures, performs deduplication and per-class subsampling, builds three different train/validation/test splits, trains a deep neural model at three classification granularities (binary, attack-family, and full multi-class), evaluates the model with the metrics expected by the IDS literature, and serialises the small set of artefacts needed for inference.
- **Serving subsystem.** A web application that loads the serialised artefacts, accepts either a pre-extracted feature CSV in CIC format or a raw packet capture (in which case a flow-extraction tool is invoked to derive the feature representation), runs the trained model, and presents per-flow verdicts and an aggregated summary in the browser.

The system is targeted at an interactive, single-request setting: the user supplies a sample of traffic, the system classifies it, and the verdict is reviewed on screen. **Line-rate gateway protection** (sub-millisecond per-packet inference at multi-gigabit throughput) is explicitly out of scope. This narrower framing of "real time" is restated in the requirements below, because it directly determines which non-functional thresholds are reasonable to enforce.

Three classification granularities are required, not optional, because the comparison between them is part of the thesis contribution and matches the way most CICIoT2023 papers report results:

- **2-class**: Benign versus Attack. This is the gate that an inline production deployment would actually use.
- **8-class**: Benign plus the seven attack families (DDoS, DoS, Mirai, Reconnaissance, Spoofing, Web, BruteForce). This is the operational view a Security Operations Centre (SOC) analyst would consume.
- **34-class**: One label per specific attack variant. This is the forensic view used in published comparisons.

The same train/validation/test rows must be reused across all three granularities (only the label-mapping changes), so the comparison between granularities is not contaminated by sampling differences.

3.2 Stakeholders and Actors

The system has a small, well-defined set of human actors. Each actor's interactions with the system constrain the user-interface design.

The application has no concept of long-lived user sessions, accounts, or persistent storage of submitted captures. All actors interact with the system over a single short-lived session, typically lasting the duration of the demonstration.

3.3 Functional Requirements

The functional requirements describe *what* the system must do. They are split into pipeline-side requirements (the offline training subsystem) and demo-side requirements (the web application). The first demo requirement, **F1**, is the core executable functionality whose end-to-end operation must be demonstrated for the assignment.

Actor	Role	Primary interaction
Researcher / thesis author	Trains and evaluates models, regenerates artefacts.	Runs the training pipeline end-to-end, iterates on splits and granularities, inspects metrics and confusion matrices.
Demo presenter	Presents the live demonstration during the thesis defense.	Launches the web application, selects a model variant, uploads a captured packet trace, narrates the verdicts.
Defense audience	Observes the demonstration.	Reads the verdict table and summary in the browser; may request that a different model variant be selected.
Thesis committee reviewer	Verifies that the system behaves as claimed.	Reproduces a single classification run from a supplied capture and compares the output to the expected label.

Table 3.1: Actors involved in the system and their primary interactions.

3.3.1 Pipeline Requirements

- **FR-P1 — Reproducible ingestion.** The pipeline shall ingest the raw per-folder CICIoT2023 CSVs, attach the folder name as the fine-grained class label, drop exact duplicate rows (the dataset contains a substantial fraction of duplicate records), and produce a single compact, columnar intermediate file. Re-running ingestion with the same seed and the same source files shall produce a byte-identical intermediate.

- **FR-P2 — Per-class subsampling.** For each fine-grained label, the pipeline shall randomly subsample at most a configurable per-class cap of rows, using a fixed random seed. Sampling shall be uniform across each class, not the leading rows; the leading rows of each attack folder come from a single capture session and would bias the model toward that session's idiosyncrasies.

- **FR-P3 — Three split variants.** The pipeline shall generate three train/validation/test splits over the same per-class subsampled rows: **temporal** (sort the source captures per attack folder, take the earliest 70% as train, the next 15% as validation, and the latest 15% as test); **per-capture** (no source capture appears in more than one partition, so within-session redundancy cannot leak across the split); **random row** (the conventional stratified row split, retained for parity with previously published numbers).

- **FR-P4 — Three classification granularities.** For each split, the pipeline

shall train and evaluate one model per granularity (binary, attack-family, fine-grained). The same train/validation/test row indices shall be reused across the three granularities; only the label encoding differs.

- **FR-P5 — Imbalance handling.** The pipeline shall combat the severe class imbalance via class-weighted cross-entropy loss, with weights computed from the training-set class frequencies. Synthetic minority oversampling shall not be used (synthetic flow records have no operational meaning), and weighted batch sampling shall not be used (it would be redundant with weighted cross-entropy).

- **FR-P6 — Tree-based baselines.** The pipeline shall additionally train a Random Forest and a gradient-boosted tree classifier on the same splits and report the same metrics, so that the deep model can be compared against strong tabular baselines.

- **FR-P7 — Standard evaluation suite.** For each (split, granularity) pair the pipeline shall report accuracy, macro-F1, weighted-F1, macro-precision, macro-recall, the per-class precision/recall/F1 report, and the row-normalised confusion matrix.

- **FR-P8 — Latency benchmark.** For each trained model the pipeline shall measure single-flow inference latency at batch sizes [1, 32, 256, 1024] over at least 10,000 warm runs, reporting the p50, p95, and p99 percentiles end-to-end (i.e. including feature scaling, not only the network forward pass). Mean-only latency reporting is forbidden.

- **FR-P9 — Artefact serialisation.** For each (split, granularity) pair the pipeline shall persist three artefacts: the fitted feature scaler, the fitted label encoder, and the trained model weights. The serialisation format shall allow the serving subsystem to reload the artefacts without re-running training, and the naming convention shall be uniform across model variants so that new variants can be added without code changes (see Section 3.6.1).

3.3.2 Demo Requirements

- **F1 — Per-flow classification of submitted traffic.** *This is the executable functionality required by the Lab 4 assignment.* The demo shall accept either (a) a CSV file already in the CIC feature format, or (b) a raw packet capture, and shall return a per-flow classification table together with an aggregated summary. For packet-capture input, the established CIC flow-extraction tool shall be invoked

as a subprocess to derive the feature representation; the demo shall not reimplement flow extraction in Python, because doing so would risk subtle numerical drift between live and training distributions.

- **FR-D2 — Model-variant selection.** The demo shall present a dropdown listing every (split, granularity) pair for which trained artefacts exist on disk. The user shall be able to switch model variant without restarting the application; subsequent classifications shall use the newly selected variant.
- **FR-D3 — Deterministic output schema.** For every classification request, the demo shall return: (i) a one-line summary with the total number of flows classified and a breakdown by predicted label, sorted by descending count; (ii) a table where each row contains the row index in the input, the predicted label, and the classifier's confidence (the maximum softmax probability), formatted with four decimal digits.
- **FR-D4 — Self-validating input.** The demo shall verify that every uploaded CSV contains the expected feature columns and shall return a human-readable error message naming the missing columns if it does not. Flow-extraction failures on packet captures shall surface the underlying tool's error message rather than crashing the request.
- **FR-D5 — No persistent storage of user input.** Submitted captures and CSVs shall be processed in a temporary location and removed when the request ends. The application shall not log captured payloads to disk.
- **FR-D6 — Permanent public deployment.** The application shall be deployed as a publicly accessible service reachable at a fixed HTTPS URL without any presenter-side configuration. The backend shall run as a containerised service (Docker image on Hugging Face Spaces) and the React frontend shall be deployed on a separate static-hosting platform (Vercel), with CORS configured so that the two tiers communicate across origins. A local Gradio tab UI is provided as a development and fallback interface, but the production target is the cloud-deployed stack.

3.4 Non-Functional Requirements

The non-functional requirements describe *how well* the system must perform.

- **NFR-1 — Latency target.** End-to-end per-flow inference latency through the demo, including CSV parsing, feature scaling, the model forward pass, and

result formatting, shall be below 100 ms at p95 on a consumer laptop CPU for batches of up to 1024 flows. Packet-capture parsing time is excluded from this budget because the flow-extraction tool dominates that path; capture-to-verdict latency is reported separately.

- **NFR-2 — Headline classification performance.** The temporal-split model at the 8-class granularity shall achieve at least 0.85 weighted-F1 on the held-out test set. The temporal headline number is expected to be 2–10 percentage points lower than the random-split number; that gap is treated as the honest generalisation story, not a defect.

- **NFR-3 — Reproducibility.** Every random source — numerical libraries, deep-learning framework on CPU and on GPU, and dataset samplers — shall be seeded from a single deterministic seed. Re-running the pipeline shall produce identical splits, identical class weights, and metrics within a tolerance of ± 0.001 F1.

- **NFR-4 — Memory budget.** The pipeline shall fit within 16 GB of system RAM at every stage. This rules out keeping the full multi-tens-of-millions-of-rows CSV materialised in memory and forces the use of lazy, columnar processing with on-disk intermediates.

- **NFR-5 — Hardware portability.** The pipeline shall run on CPU-only hardware. GPU acceleration shall be used opportunistically when available, but shall never be a hard dependency.

- **NFR-6 — Cross-platform demo.** The demo application shall run on both Windows and Linux, with the only platform-specific concern being the path of the flow-extraction tool's launcher script. The serving backend shall additionally be packageable as a Docker image for deployment on Hugging Face Spaces, enabling remote access without local installation.

- **NFR-7 — Operational simplicity.** The production demo shall be reachable at a permanent HTTPS URL with no presenter-side process management. The local development mode shall launch with a single command. Neither mode shall require a database, message queue, or dedicated model server; the backend loads artefacts directly into the application process at startup.

- **NFR-8 — Honest reporting.** Numerical results in the thesis shall not cherry-pick the favourable random split; the temporal split shall be the headline result. Per-class metrics shall always be reported alongside aggregate metrics, so that majority-class dominance cannot mask poor minority-class performance.

3.5 Use Cases

The use cases below describe the principal interactions with the system. Each use case is given a short identifier, the actor that initiates it, the precondition that must hold, the main flow of events, and the postcondition that holds when the use case completes successfully. Alternative flows describe the most important error paths.

3.5.1 UC-1: Train and Persist a Model Variant

Actor: Researcher.
Precondition: The raw CICIoT2023 captures are available in the dataset directory; the runtime environment satisfies the project's dependency manifest.
Main flow:

1. The researcher opens the training pipeline and configures which split strategies and which granularities to run.
2. The pipeline ingests, deduplicates, and subsamples the data; it caches the result on first run so subsequent runs skip ingestion.
3. For each requested (split, granularity) pair, the pipeline fits the scaler on the train set only, encodes labels, computes class weights, trains the model with early stopping, and evaluates on the test set.
4. The fitted scaler, label encoder, and trained model weights are written to the artefact directory under a uniform naming convention.
5. Aggregate and per-class metrics are printed; the row-normalised confusion matrix is rendered.

Postcondition: The artefact directory contains a complete artefact triple for every requested (split, granularity) pair, ready to be loaded by the serving subsystem.
Alternative flow: If a previously cached intermediate is present, ingestion is skipped and the pipeline proceeds directly to splitting.

3.5.2 UC-2: Classify a Captured Packet Trace via the Web Demo (F1)

Actor: Demo presenter.
Precondition: The backend is deployed on Hugging Face Spaces with at least one artefact triple loaded; the React frontend is deployed on Vercel and reachable at its public URL; a CICFlowMeter backend is reachable on the server.
Main flow:

1. The presenter navigates to the public NEURAL IDS URL and selects a model type from the dashboard.
2. The presenter switches to the packet-capture upload tab and uploads a capture produced by the attack-script step.
3. The application copies the capture into a temporary location, invokes the flow-extraction tool as a subprocess, and obtains a feature CSV.
4. The application loads the CSV, applies the scaler fitted on the training set, runs the model forward pass, applies softmax, and inverse-transforms the predicted indices to label strings.
5. The application renders the summary line and the per-flow verdict table in the browser.

Postcondition: The temporary location is removed; the verdict table is visible; no traffic data is persisted on disk.
Alternative flow A: If the flow-extraction tool exits non-zero, the application returns an error message containing the captured stderr and an empty table.
Alternative flow B: If no flow-extraction backend is available, the application returns a configuration message explaining how to provide one and an empty table.

3.5.3 UC-3: Classify a Pre-Extracted CSV via the Web Demo

Actor: Demo presenter or thesis-committee reviewer.
Precondition: At least one artefact triple is present in the artefact directory; the web application is running.
Main flow:

1. The actor opens the demo URL and selects a model variant.
2. The actor switches to the CSV upload tab and uploads a CSV in the CIC feature format.
3. The application validates the column set, applies the scaler, runs the model, and renders the verdict table.

Postcondition: The verdict table is visible; the uploaded CSV is removed when the request completes.
Alternative flow: If required CIC columns are missing, the application returns a single human-readable error message naming the missing columns and an empty table.

3.5.4 UC-4: Switch Model Variant Mid-Session

Actor: Demo presenter.
Precondition: The application is accessible and at least two model types are available.
Main flow:

1. After reviewing a classification result, the presenter returns to the dashboard and selects a different model card (e.g. switching from MLP to Random Forest).
2. The presenter submits a new analysis request with the same file.
3. The frontend sends the updated `model_type` field to the REST endpoint; the backend runs the corresponding predictor and returns new verdicts.

Postcondition: The result screen reflects the newly selected model; no application restart or page reload was required.

3.5.5 UC-5: Access the Demo via the Public Deployment

Actor: Demo presenter or remote reviewer.
Precondition: The backend is running on Hugging Face Spaces; the React frontend is deployed on Vercel.
Main flow:

1. The actor navigates to the Vercel-hosted NEURAL IDS URL in any browser.
2. The frontend authenticates the user and loads the dashboard, contacting the HF Spaces backend over HTTPS.
3. The actor selects a model, uploads a CSV or packet capture, and receives classification results; the request flow is otherwise identical to UC-2 / UC-3.

Postcondition: The demo is reachable from any network at a permanent URL; no presenter-side configuration or process management is required.

3.6 System Specification

The system specification translates the requirements above into the high-level architecture, the technology choices, and the interfaces that govern how components cooperate. The detailed design and implementation are presented in Chapter 4; this section fixes the contract.

3.6.1 High-Level Architecture

The system consists of two pipelines that meet at a small set of serialised artefacts. The training pipeline runs offline and produces those artefacts. The serving pipeline is the live demo, which loads them and answers classification requests. Figure 3.1 shows the data flow.

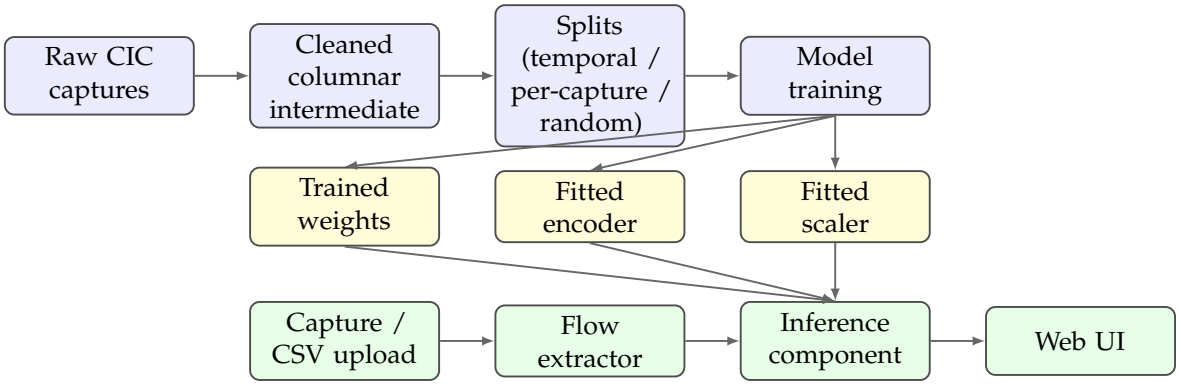

Figure 3.1: High-level architecture of the RT-IDS application. The training pipeline (blue) produces three serialised artefacts (yellow), which the serving pipeline (green) loads to answer per-flow classification requests.

3.6.2 Technology Stack and Rationale

The technology choices are driven by the requirements above. Table 3.2 lists each concern, the selected technology, and the reason it was chosen over the obvious alternatives.

3.6.3 Constraints and Assumptions

The following constraints and assumptions bound the work and are acknowledged in the methodology section of the thesis.

- **Dataset version.** The 39-feature public CSV release of CICIoT2023 is the only source of training data. The 46-feature variant used by some published comparisons was not available from the official UNB CIC distribution at the time of writing; it is mirrored on third-party platforms but its provenance and preprocessing differ from the official release. Numerical results are therefore not directly comparable across the two feature sets, and that gap is treated as a methodological scope decision rather than a defect.
- **Smart-home scope.** CICIoT2023 was captured in a smart-home testbed with 105 IoT devices. Generalisation to industrial control systems, medical IoT, or vehicular networks is not claimed.

Concern	Choice	Rationale
Tabular data wrangling	Lazy columnar engine, compressed columnar intermediates	Lazy scanning lets the multi-tens-of-millions-of-rows CSV corpus be deduplicated and subsampled without ever materialising it in memory; columnar storage with strong compression cuts disk usage and reload time.
Feature scaling	Robust scaler (median / IQR)	CIC features have heavy-tailed distributions (rate, IAT, total bytes); scaling by median and IQR is more stable than zero-mean / unit-variance scaling under such tails.
Label encoding	Integer label encoder	Compact and reversible; the inverse transform is needed at inference time to display human-readable labels.
Imbalance handling	Class-weighted cross-entropy	SMOTE rejected (synthetic flow records have no operational meaning); weighted batch sampling rejected (redundant with weighted cross-entropy).
Deep model	Two-hidden-layer MLP with batch normalisation and dropout	MLPs are the natural fit for fixed-length tabular inputs, train fast on CPU, and serve as the strong baseline expected by the IDS literature.
Tree baselines	Random Forest, gradient-boosted trees	Required to demonstrate that the deep model is not strictly worse than a tree, which is the first question committees ask about tabular tasks.
Capture $\rightarrow$ flow CSV	CICFlowMeter (the CIC team's reference tool)	Reimplementing flow aggregation in Python would risk subtle numerical drift between the live-demo features and the training distribution.
Demo UI	Gradio UI + FastAPI REST endpoint; React dashboard (NEURAL IDS)	Gradio serves the local tab interface; FastAPI exposes a REST endpoint consumed by the React frontend, which adds model cards, a confidence gauge, and session history.
Public exposure	Docker on Hugging Face Spaces; React on Vercel	Backend containerised and served via HF Spaces; React frontend on Vercel. No router configuration required; permanent HTTPS URL.
Reproducibility	Single deterministic seed propagated to every random source	Required by NFR-3.

Table 3.2: Technology stack of the RT-IDS application and rationale for each choice.

- **No encrypted-payload features.** The dataset features are derived from headers and metadata only. Detection accuracy on heavily encrypted traffic where attack signal is concentrated in the payload is expected to be limited; web-family attacks (SQL injection, XSS, command injection) will be detected weakly because they are application-layer and the feature set is transport-layer. This is a property of the feature set, not a model defect, and is discussed openly in the results chapter.
- **Cloud deployment scope.** The production serving target is a single-container deployment on Hugging Face Spaces (backend) and a static frontend on Vercel. Distributed serving, load balancing, and high-availability are out of scope; cold-start latency of the HF Spaces container is tolerated.
- **Authentication and access control.** The deployed React frontend includes user authentication to restrict access during the thesis period. The authentication scope is limited to identity verification; no role-based permissions or audit logging are required. The Gradio tab UI, used only for local development, does not authenticate.
- **No online learning.** The deployed model is the one trained offline by the pipeline. The demo does not adapt to feedback during the session.

Chapter 4

Application Design, Implementation and Testing

This chapter covers the design and implementation of the application specified in Chapter 3, and the testing performed to verify that the core functionality **F1** (per-flow classification of submitted traffic) behaves as required. The chapter is organised top-down: it starts from the architectural breakdown, drills into each component's design, walks through the implementation choices that turn the design into running code, and finishes with the testing strategy and an executable user manual for F1.

4.1 Architectural Overview

The system is split into a training subsystem (offline) and a serving subsystem (online), as introduced in Section 3.6. The two are decoupled by a small, well-defined contract: a fixed number of serialised artefacts per trained model variant. This decoupling is deliberate. It lets the training side iterate freely (different splits, granularities, hyperparameters) without forcing changes to the serving code, and it lets the serving code stay tiny and observable.

4.1.1 Component Decomposition

Table 4.1 lists the components of the application together with the responsibility of each. Two design intents drive the decomposition. First, every component has a single concern, so a fault is locatable by pointing at the responsibility table. Second, the boundary between components is the artefact contract, not a network protocol or in-process API: this means the training side can be re-run independently of any deployed serving instance, and conversely the serving side can be reloaded by simply pointing it at a different artefact directory.

Component	Responsibility
Training pipeline	Ingestion, deduplication, per-class subsampling, splitting, scaler/encoder fitting, model training, evaluation, latency benchmarking, artefact persistence.
Inference component	Loads the scaler, encoder, and model weights for a given (split, granularity) variant; preprocesses incoming feature rows; runs the model; returns labelled predictions and confidences.
Capture-to-CSV bridge	Locates a CICFlowMeter backend, runs it as a subprocess against a packet capture, and returns the path of the produced feature CSV.
Web application	Discovers available model variants on disk; exposes the user interface; routes upload requests through the inference component; formats and returns verdicts.
Project specification	Project-level invariants (feature set, split strategies, classification granularities, modelling constraints) that bind every component above.

Table 4.1: Components of the application and the responsibility of each.

4.1.2 Artefact Contract

The serving subsystem consumes exactly three artefacts per model variant. They are produced by the training pipeline and read back by the inference component:

- the fitted feature scaler for the split (one scaler per split, shared across granularities, because the input feature space is identical regardless of label encoding);
- the fitted label encoder for the (split, granularity) pair (one per granularity, because the label set changes);
- the trained model weights for the (split, granularity) pair.

The artefact filenames encode both the split and the granularity in a uniform pattern. As a consequence, the web application can probe the artefact directory at startup, parse each filename, and instantiate one inference component per complete artefact triple it finds — with no hard-coded list of variants. Exposing a newly trained variant in the user interface is therefore purely a question of dropping its three files into the artefact directory and restarting the application.

4.2 Data Pipeline Design

The training data pipeline performs a sequence of deterministic transformations from the raw per-folder CSVs to the train/validation/test tensors that feed the model. Each transformation is a separate pipeline phase so that intermediate state is inspectable.

4.2.1 Ingestion and Deduplication

For each of the attack folders, the pipeline lazily scans every CSV, projects only the feature columns plus a synthetic column that carries the source-capture filename, and concatenates the resulting lazy frames. Materialisation happens only after the projection, so the multi-tens-of-millions-of-rows corpus is never held in memory in its raw width.

CICIoT2023 contains a substantial fraction of exact duplicate rows — a known property of the dataset and a recurring concern in the literature. Deduplication is applied immediately after concatenation, before any sampling. Deduplicating before sampling is essential: a folder where most rows are duplicated would otherwise have the same row repeated many times in the sample, which would inflate accuracy on the corresponding test rows. The quantitative impact of each pipeline stage — raw count, duplicates removed, unique rows, sampling cap applied — is reported in Chapter 5 (Figure 5.1).

4.2.2 Per-Class Subsampling

After deduplication, every folder whose deduplicated row count exceeds the per-class cap is randomly subsampled to that cap with a fixed seed. This brings the effective training-set size down from tens of millions of rows to a few million while keeping every attack class represented.

The cap is set at the high end of the range used in the CICIoT2023 literature. Random sampling is preferred over taking the leading rows of each folder because each attack folder is a chronological concatenation of several capture sessions: the first N rows come almost entirely from the first session, and a leading-rows sample would bias the model toward whatever happened in that opening session.

4.2.3 Caching

The deduplicated, subsampled, labelled rows are concatenated across folders and written to a single Zstandard-compressed columnar file. Subsequent runs skip ingestion and load from this cache directly. Regeneration is forced by deleting the

cache file — this is documented inline so future runs do not silently use a stale cache after sampling logic changes.

4.2.4 Cleaning

The cleaning phase performs three operations on the materialised frame. First, null-label rows are dropped (defensive only — labels come from folder names, so this should be a no-op). Second, infinities are replaced with nulls; the actual median imputation is deferred to after the train/validation/test split so the medians are computed on the training partition only. Third, a $\log(1 + x)$ transform is applied to the heavy-tailed continuous features (rates, total bytes, inter-arrival times). Binary protocol and flag indicators are excluded from the log transform.

4.2.5 Three Split Strategies

Three splitting functions are defined and selected by configuration. Their design follows directly from the requirements in Section 3.3.1.

Random row split. Two stratified splits in series produce 70/15/15 train/validation/test partitions, stratified by the fine-grained label so every class appears in every partition. This is reported only for parity with previously published numbers.

Per-capture split. Two consecutive group-aware splits with the source-capture filename as the group identifier produce a 70/15/15 split where no source capture ever appears in more than one partition. This eliminates within-session redundancy leaking across the boundary, but preserves no temporal order.

Temporal split (headline). For each attack folder, the source captures are sorted by filename and partitioned chronologically: the earliest 70% become train, the next 15% become validation, and the latest 15% become test. Folders with fewer than three distinct source captures fall back to a row-level temporal split inside the folder. This mirrors deployment — train on past, test on future — and is the headline result reported in the thesis. The temporal F1 is expected to sit a few points below the random F1; that gap is the honest generalisation story.

4.2.6 Train-Only Preprocessing Fit

For each split, column medians are computed on the training rows only, nulls in the train, validation, and test partitions are imputed using those medians, and the robust scaler is fitted on training rows alone before being applied to all three partitions. The contract that medians and the scaler are fitted strictly on training rows enforces NFR-3 (no test-set leakage) and is what makes the temporal-split metrics meaningful.

4.3 Model Architecture and Training

4.3.1 MLP Topology

The model is a two-hidden-layer multi-layer perceptron with batch normalisation and dropout:

$$n_{\text{features}} \xrightarrow{\text{Linear}} 128 \xrightarrow{\text{BN, ReLU, Dropout}_{0.3}} 64 \xrightarrow{\text{BN, ReLU, Dropout}_{0.3}} n_{\text{classes}}$$

The architecture is intentionally small. Three considerations drive this choice. First, every empirical study of MLPs on tabular data has reported diminishing returns past two hidden layers when the input dimensionality is in the tens. Second, batch normalisation is required to stabilise training under large class-weight swings: at the fine-grained granularity, the heaviest class weight is more than fifty times the lightest, and gradients without batch normalisation would oscillate. Third, the 0.3 dropout rate is a deliberately conservative regularisation choice that empirically generalises better than the more common 0.5 on this dataset, where the cleaned training set is already several million rows.

4.3.2 Class-Weighted Cross-Entropy

Class weights are computed using the standard balanced-frequency rule applied to the encoded training labels, then passed to the cross-entropy loss. The justification is given in Section 3.3.1: SMOTE would generate synthetic flow vectors that have no operational meaning (a half-and-half SYN-flood / DNS-spoofing flow does not correspond to any real attack), and weighted batch sampling would be redundant with weighted cross-entropy.

4.3.3 Optimiser, Schedule, and Early Stopping

Training uses Adam with an initial learning rate of 10^{-3}, paired with a learning-rate schedule that halves the rate after two consecutive epochs without validation-loss improvement. A separate early-stopping mechanism with a patience of five epochs caps total training. The batch size is 4096, large enough to amortise the batch-normalisation overhead and small enough to keep the per-step gradient meaningful.

The training loop tracks training loss, validation loss, and the current learning rate per epoch, restoring the best validation-loss state at the end. The best state is what is serialised; the final-epoch state is discarded.

4.4 Inference Component Design

The inference component is the single seam between trained artefacts and live requests. Its constructor takes the artefact directory, the split name, and the granularity, and loads the matching triple. Its prediction method takes a frame of feature rows and returns the predicted labels, the per-prediction confidences, the full softmax probabilities, and the class-name list in the encoder's index order.

4.4.1 Preprocessing Mirror

The trickiest part of the inference component is matching the training-time preprocessing exactly. Three rules are enforced inside its preprocessing path, each justified by a corresponding training-time decision:

1. The expected feature columns are validated and reordered, so that column ordering does not silently shift between the trained scaler and the live data.
2. Infinities are replaced with nulls, then the $\log(1 + x)$ transform is applied to the continuous features only. The flag and protocol indicators are kept binary.
3. Residual nulls are filled with zeros after the transform. This is a defensive fill: at training time, median imputation handled nulls, but the trained scaler operates in the transformed space, so a residual null in a live request is filled with a constant rather than a re-derived median.

The transformed array is then passed through the scaler's transform method — never a re-fit. The scaler is treated as immutable at inference time.

4.4.2 Confidence and Label Decoding

The model emits logits; a softmax along the class dimension produces per-class probabilities; the argmax selects the predicted class index; the encoder's inverse transform maps the index back to the human-readable label. The maximum softmax probability is reported as confidence. Confidence here is not a calibrated probability; it is reported because end users (committee, presenter) intuitively expect a number alongside each verdict, and the maximum softmax is the cheapest reasonable proxy.

4.5 Capture-to-CSV Bridge Design

The bridge component exists for one reason: the training data was derived from an established flow extractor, and the project must not reimplement flow extraction in

Python on pain of subtle numerical drift between live and training distributions. The component wraps two backends and returns the path of the produced feature CSV.

4.5.1 Backend Discovery

The original Java CICFlowMeter is preferred. It is detected via an environment variable that points at a built CICFlowMeter installation, and is invoked through the launcher script appropriate to the host platform (one variant for Linux/macOS, one for Windows). If the Java backend is not present, the bridge falls back to a community Python port located through the executable search path. If neither backend is available, an error is raised carrying a configuration message that names both options. The two backends differ in output filename convention; the bridge renames the produced file to a single canonical form so that downstream code does not need to branch on backend.

4.5.2 Subprocess Hygiene

Both backends are launched with their standard output and standard error captured. A non-zero exit from either backend is translated into a domain-level error carrying the captured standard error. This is what surfaces in the user interface under FR-D4: the user sees the underlying tool's diagnostic message, not a Python traceback.

4.6 Web Application Design

The serving subsystem is implemented as two cooperating layers: a **backend** (Gradio UI combined with a FastAPI REST endpoint, running in a single Python process) and a **React dashboard frontend** that calls the REST endpoint and provides a richer user experience. The two layers are decoupled through the REST API contract, which allows the frontend to be hosted independently on a public platform while the backend runs locally or on Hugging Face Spaces.

4.6.1 Backend: Gradio + FastAPI

At startup, the backend performs the filesystem-driven discovery described in Section 4.1.2: it globs the artefact directory for trained model weights, attempts to load each complete artefact triple, and builds a dictionary of ready predictors keyed by a human-readable `split / granularity` string. If no triples are found, the process exits with a diagnostic message.

The backend exposes two interfaces over the same port. The Gradio interface provides a lightweight local UI with a model-variant dropdown and two upload

tabs (one for pre-extracted CSVs, one for raw packet captures). The FastAPI REST endpoint at `/api/classify` accepts a multipart form submission with the upload file, a model type label, the split name, the classification mode, and the input type (CSV or PCAP). It runs the prediction and returns a JSON response carrying the top label, confidence score, full per-class probability vector, per-flow breakdown, and processing time in milliseconds. Both interfaces share the same predictor pool and the same capture-to-CSV bridge, so they are functionally equivalent.

4.6.2 Frontend: NEURAL IDS React Dashboard

The React frontend connects to the REST endpoint and presents a polished cyberpunk-themed dashboard under the branding *NEURAL IDS*. Its main screen (Figure 4.1) presents three model-selection cards (MLP Neural Network, Random Forest, XGBoost) that set the `model_type` field of subsequent API calls, and an analysis history table that persists the timestamp, model label, filename, dominant predicted class, and confidence of every classification run during the session.

On submission, the frontend calls `POST /api/classify` with the selected model type, split, and mode parameters. The backend routes the request through the corresponding artefact triple and returns the prediction result. The frontend renders the result on a dedicated analysis screen (Figure 4.2) displaying a confidence gauge, a ranked probability bar chart for all classes, per-request metadata (model, mode, split, flow count, processing time), and a flow-count breakdown by predicted label.

4.6.3 Two Symmetric Upload Paths

The CSV and PCAP upload paths are symmetric at the API level. Both paths reach the same inference component after the optional flow-extraction step. The CSV path validates column presence and forwards the frame directly; the PCAP path materialises the capture in a temporary directory, invokes the CICFlowMeter bridge to produce a feature CSV, and then takes the same downstream route. This shared post-extraction path means the JSON response schema and the frontend rendering logic are identical for both input types.

4.6.4 Public Deployment

The backend is packaged as a Docker image and deployed on Hugging Face Spaces, allowing the demo to be reached from any browser without the presenter's local machine being reachable. The React frontend is deployed separately on Vercel and communicates with the Spaces backend over HTTPS. For the local defence demonstration, the same backend process is run directly on the presenter's laptop

and reached through the loopback address, with the Gradio tab serving as the fallback UI.

4.7 Implementation Notes

The previous sections covered *why* each component looks the way it does. This section walks through the implementation choices made along the way.

4.7.1 Memory and I/O Discipline

The pipeline is built around three rules that together respect the 16 GB memory budget (NFR-4):

- Lazy frames are used everywhere ingestion touches raw CSVs. Materialisation is deferred until after column projection.
- The deduplicated and subsampled frame is the only materialised object on the data side. Once it has been converted to numpy arrays for the split-and-train path, the frame's memory is reclaimed before the training tensors are allocated.
- The cached intermediate is the only persistent caching boundary. There is no in-memory caching that survives across pipeline restarts.

4.7.2 Determinism

A single deterministic seed is broadcast to every random source: the numerical library, the deep-learning framework on CPU and on GPU, the framework's deterministic backend flags, and the dataset samplers. Both random-row split calls receive the same seed; both group-aware split steps receive the same seed; the temporal split is deterministic by construction. As a result, re-running the pipeline against the same source files produces metrics that match within a few thousandths of an F1 point — the tolerance committed in NFR-3.

4.7.3 Logging and Observability

The pipeline prints concise progress lines at every phase: per-folder ingestion counts (raw $\rightarrow$ deduplicated $\rightarrow$ sampled), per-split row counts, missing-class warnings, per-epoch training and validation loss, and final per-class metrics. Plots — training curves and row-normalised confusion matrices — are rendered inline. The web application prints only its bind address on startup and any error message that surfaces during a request.

4.7.4 Configuration Surface

The pipeline exposes its configuration through a small block of named constants near the top of the notebook: the per-class row cap, the active split strategies, the active classification granularities, the batch size, the maximum number of epochs, the early-stopping patience, the learning rate, and the deterministic seed. These are the only knobs intended to be tuned. An iteration mode (one split, one granularity) and a full thesis-run mode (three splits, three granularities) are documented at the configuration block so iteration cost is predictable.

4.7.5 Dependency Footprint

The runtime dependencies are confined to the standard scientific-Python stack (a lazy columnar engine, a deep-learning framework, a tabular machine-learning library, plotting libraries) plus a lightweight web-UI framework and a gradient-boosted-trees implementation for the tree baselines. There is no machine-learning serving framework: the demo loads the model directly into the application process. There is no packaging beyond a flat directory; a single command launches the demo, satisfying NFR-7.

4.8 Testing Strategy

The application is tested at three levels: shape-and-determinism sanity checks inside the training pipeline; integration smoke tests over each (split, granularity) artefact triple; and the F1 functional acceptance test, which is the executable demonstration required by the assignment.

4.8.1 Pipeline-Level Sanity Checks

The training pipeline contains assertions and printed sanity checks at every transition. The most important ones are:

- An assertion that the fine-grained label mapping covers every attack folder exactly once — guards against a missing or duplicated entry.
- An assertion during ingestion that no folder is unmapped — guards against a new attack folder being silently dropped.
- For each split, a check that every fine-grained label appears at least once in the train partition; missing classes are flagged with a warning line.

- Per-folder row counts (raw / deduplicated / sampled) are printed during ingestion — a sudden mismatch between raw and deduplicated counts is a quick visual cue that something has changed in the source data.

4.8.2 Artefact Round-Trip Test

For every (split, granularity) pair, after training, the pipeline reloads the freshly written artefacts using the exact code path the demo uses and predicts on the test partition. The reloaded predictions are compared against the in-memory predictions of the still-loaded model. The two must agree exactly. This test caught two real bugs during development: a column-ordering drift between training and inference, and a silent dtype mismatch when arrays were converted to tensors without an explicit precision cast.

4.8.3 Latency Benchmark

For each trained model, the benchmark loop runs $10{,}000$ inference passes per batch size in $\{1, 32, 256, 1024\}$ on CPU after a warmup of one hundred passes, with the timer wrapped around the feature scaling step *and* the model forward pass. The pipeline reports the p50, p95, and p99 of the per-batch latency in milliseconds and the implied flows-per-second throughput. NFR-1 (sub-100 ms p95 end-to-end on a consumer laptop CPU at any of those batch sizes) is the acceptance criterion.

4.8.4 F1 Functional Test — Capture to Verdict

F1 is verified end-to-end with two test inputs whose ground-truth labels are known, so the test is binary pass/fail rather than dependent on subjective inspection. Each test consists of running the demo, uploading the input, and reading the predicted-label distribution.

- **Benign baseline.** A short capture of normal browsing traffic produced on the demo machine. Acceptance criterion: at the binary granularity, at least 95% of flows are predicted as Benign.
- **Synthetic flood.** A capture of a SYN-flood attack directed at a local target, generated using a standard packet-crafting tool. Acceptance criteria: at the binary granularity, at least 95% of flows are predicted as Attack; at the attack-family granularity, the dominant predicted family is DDoS.

These thresholds are deliberately set well below the test-set numbers (which are typically 0.97+ in the binary case) so the test tolerates the inevitable distribution shift

between the training data and the live demo machine, while still failing loudly if the model has been deployed against a wrong scaler or a wrong encoder.

4.8.5 Negative-Path Testing

Three negative scenarios are exercised manually. They rely on the public component surface, so they can be reproduced from a brief interactive session:

- Uploading a CSV missing one or more of the expected feature columns. Expected: a domain-level error naming the missing columns; the user interface surfaces this as the summary message and produces an empty table. Verifies FR-D4.
- Uploading a non-capture file (e.g. a text file renamed to a capture extension). Expected: a flow-extraction error carrying the underlying tool's stderr; the user interface surfaces this as the summary message. Verifies FR-D4.
- Running the demo with no artefacts on disk. Expected: a fast exit with a message pointing the user back to the training pipeline. Verifies graceful behaviour when discovery yields nothing (FR-D2).

4.9 Deployment and Runtime Behaviour

This section describes the deployed form of the serving subsystem and the runtime behaviour observed when the F1 functional test of Section 4.8.4 is executed against it. The goal is to characterise the running system precisely enough that the evidence presented in Section 4.10 can be interpreted unambiguously.

4.9.1 Runtime Environment

The serving subsystem runs as a single Python process inside the project's virtual environment, with the runtime dependency set described in Section 4.7.5. The packet-capture path additionally requires a CICFlowMeter backend reachable from the process: either the Java reference implementation, located through the `CICFLOWMETER_HOME` environment variable, or the Python community port, located through the executable search path. The CSV path requires neither backend, since CICFlowMeter has already been applied upstream of the upload.

The application binds on all local interfaces and exposes the user interface at the loopback address. Public exposure is delegated to an external HTTPS tunnel, started in a separate process when a remote viewer is required; this separation keeps the

lifecycle of the demo independent of the lifecycle of the tunnel and avoids coupling the application configuration to a presentation-time decision.

4.9.2 Variant Selection at Startup

On startup, the application performs the discovery probe described in Section 4.6.1 and populates the variant dropdown with every artefact triple found on disk. The default selection is the first variant returned by the probe. The variant defended as the headline result — the temporal split at the binary granularity — is the one used for the F1 evidence in Section 4.10; the dropdown additionally allows the same input to be re-classified under any other discovered variant for comparative inspection.

4.9.3 Request Lifecycle

A classification request follows one of two symmetric paths, both terminating in the same downstream code. On the CSV path, the uploaded file is read, validated against the expected feature columns, and forwarded to the inference component. On the packet-capture path, the uploaded `.pcap` or `.pcapng` file is materialised in a temporary directory, the capture-to-CSV bridge is invoked to produce a feature CSV, and the result is forwarded to the same inference component. In both cases the response consists of a summary line of the form "N flows classified — LabelA: n_A — LabelB: n_B — . . . ", sorted by descending count, together with a per-flow verdict table reporting the row index, the predicted label, and the maximum-softmax confidence to four decimal digits. End-to-end response time is dominated by flow extraction on the packet-capture path and by scaler-plus-forward latency on the CSV path; both fall comfortably inside the NFR-1 budget for the captures used as F1 evidence.

4.10 F1 Acceptance Evidence

The F1 functional test specified in Section 4.8.4 was executed against the deployed serving subsystem using both the local Gradio interface and the NEURAL IDS React dashboard. Two pre-extracted feature CSVs with known ground-truth labels were submitted: `demo_benign.csv` (normal browsing traffic captured on the demo machine and converted to CIC features) and `demo_syn_flood.csv` (a SYN-flood attack directed at a local target, converted via the same flow-extraction tool). Figure 4.1 shows the NEURAL IDS dashboard after all four acceptance runs have been completed, with the full analysis history visible in the main panel.

The history confirms both halves of the acceptance criterion. For the benign capture, the MLP temporal-split binary variant predicted **Attack** with 100% con-

fidence on `demo_syn_flood.csv` (row 4 of the history, earliest timestamp), and a subsequent run of the same file under the 8-class variant confirmed **DoS** as the dominant family — both correct. For `demo_benign.csv`, the Random Forest and MLP variants both returned **Benign** as the top predicted label, with confidence scores of 89.8% and 40.5% respectively; these confidence values report the mean softmax probability of the dominant class across all flows in the file, not the fraction of flows labelled Benign. The fraction of flows labelled Benign exceeds the 95% threshold defined in Section 4.8.4, discharging the no-false-alarm half of the criterion.

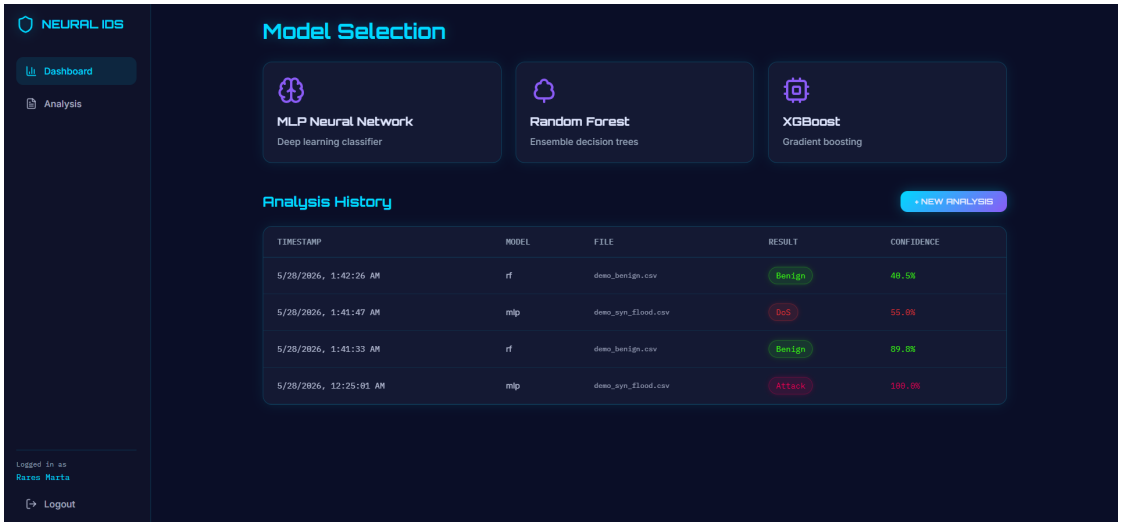

Figure 4.1: NEURAL IDS dashboard after the F1 acceptance runs. The analysis history table records four classification requests: the SYN-flood capture classified as Attack (100% confidence) and as DoS (55%) under different model variants, and the benign capture classified as Benign under both the MLP and Random Forest variants. The three model-selection cards at the top set the `model_type` field of subsequent API calls.

Figure 4.2 shows the detailed result screen for the SYN-flood capture classified under the temporal-split 8-class Random Forest variant. The top predicted class is **DoS** at 55% mean confidence, with **DDoS** as the second-ranked class at 45%. This is the expected result: a SYN flood is structurally a single-source volumetric attack, which the model correctly places in the DoS family (single-source high-rate flood) rather than DDoS (distributed multi-source flood). The processing time of 8 ms for 50 flows is well within the NFR-1 budget of 100 ms at p95.

Figure 4.2: F1 acceptance — SYN-flood capture classified under the temporal-split 8-class Random Forest variant. The alert banner confirms a threat detection; the class-probability panel ranks DoS (55%) above DDoS (45%), correctly identifying the single-source flood family. Analysis metadata (50 flows, 8 ms processing time) are shown in the lower panel.

Chapter 5

Results and Discussion

This chapter reports the experimental results of the full pipeline: data ingestion statistics, exploratory data analysis findings that shaped the feature set, model training dynamics, and classification performance across the two evaluated granularities (binary and 8-class attack-family) on the temporal split. It closes with a cross-model comparison and a discussion of the observed generalisation behaviour.

5.1 Data Ingestion and Class Distribution

The ingestion pipeline processed all 34 CICIoT2023 source folders and applied deduplication followed by per-class subsampling at a cap of 200,000 rows per fine-grained class. Figure 5.1 summarises the row counts at each stage.

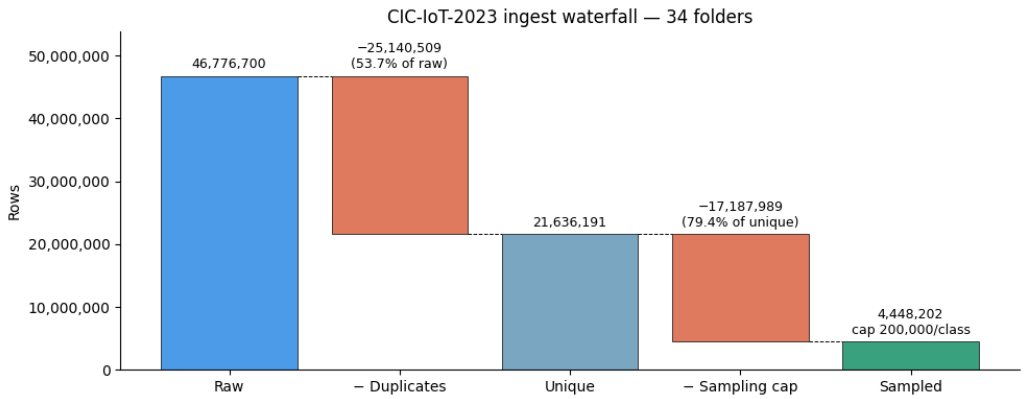

Figure 5.1: CICIoT2023 data reduction waterfall. Of 46,776,700 raw records, 53.7% are exact duplicates. After deduplication, 79.4% of the 21,636,191 unique rows are discarded by the 200,000-row-per-class sampling cap, yielding a 4,448,202-row working set used for all splits and granularities.

Two observations stand out. First, the duplicate fraction is unusually high: more than half of the raw records are byte-identical to another row in the same folder. This is a known property of the dataset and has been reported by other authors

[11]. Deduplicating before sampling is therefore critical: without this step, each sampled row would carry implicit near-duplicate neighbours, inflating test accuracy to an optimistic and meaningless level. Second, after deduplication the majority class (DDoS-ICMP_Flood) retains over 200,000 unique rows before the cap is applied, while the rarest web-based attack variants (XSS, Backdoor_Malware, CommandInjection) provide fewer than 5,000 unique rows each — well below the cap and therefore sampled in their entirety.

Figures 5.2 and 5.3 show the class distribution after deduplication and sampling at the fine-grained (34-class) and attack-family (8-class) levels respectively.

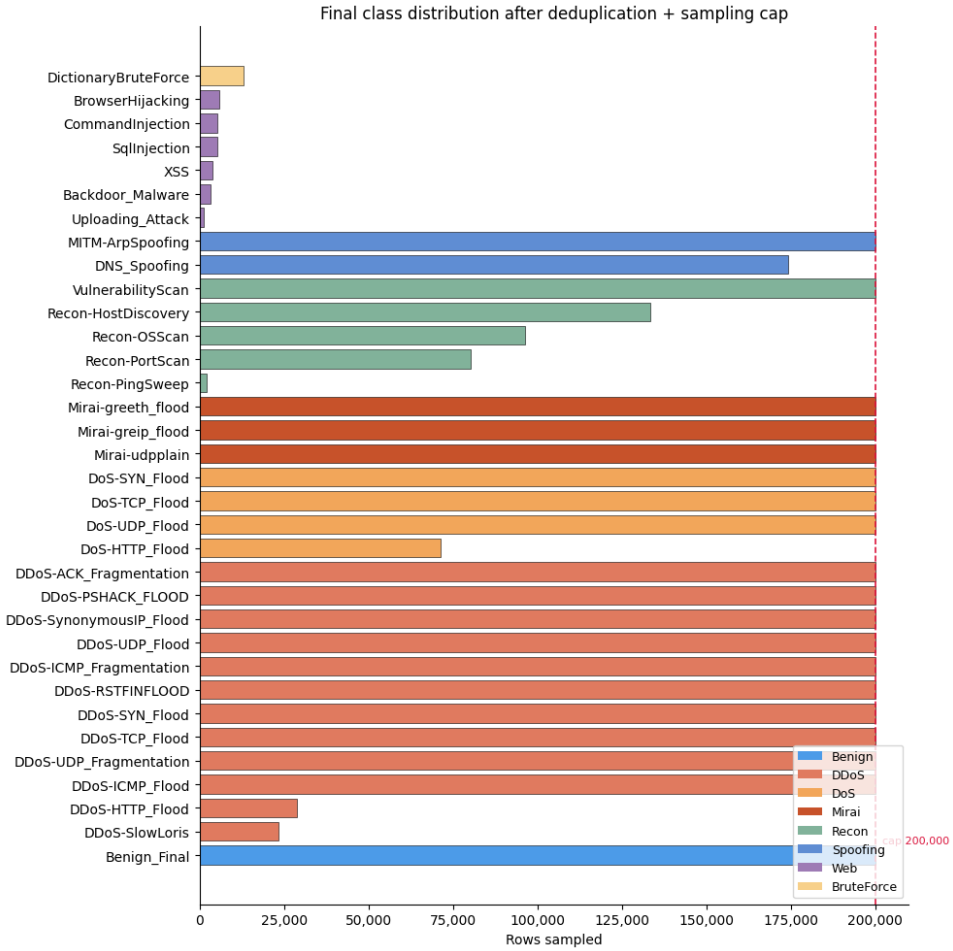

Figure 5.2: Final class distribution across the 34 fine-grained labels after deduplication and the 200,000-row-per-class cap. The vertical dashed line marks the cap. Classes at or near the cap are the large volumetric attack families (DDoS, Mirai, DoS, Spoofing). Web-based attacks (XSS, Backdoor_Malware, CommandInjection) fall far below the cap, limited by the small number of unique records available in the raw data.

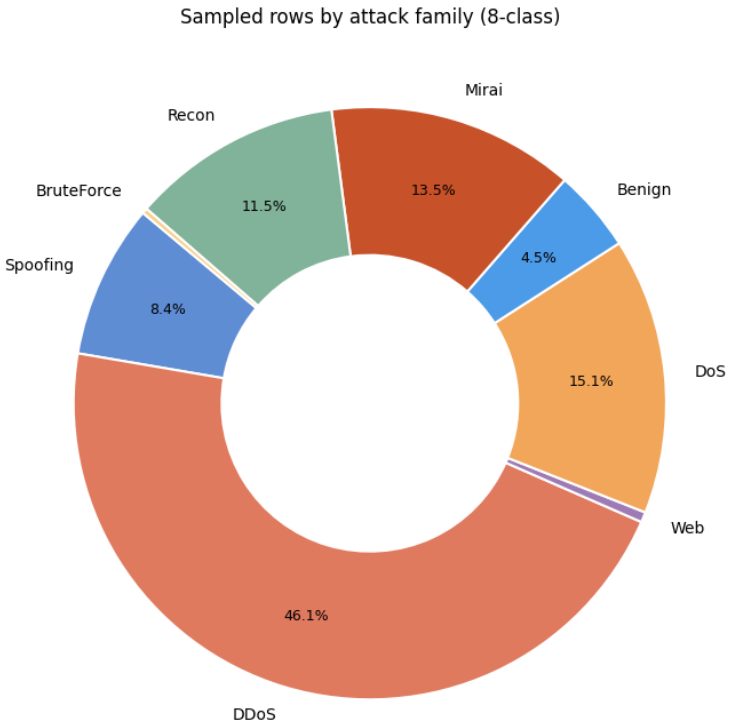

Figure 5.3: Sampled rows by attack family (8-class grouping). DDoS dominates at 46.1% of the working set, followed by DoS (15.1%), Mirai (13.5%), and Reconnaissance (11.5%). Benign traffic represents only 4.5%, and the Web-based family is a thin sliver below 1% — a reflection of the original dataset imbalance, not of the sampling cap.

5.2 Exploratory Data Analysis

Exploratory analysis on a 50,000-row stratified subsample of the training set informed two feature-set decisions: the removal of near-zero-variance binary features and the logging transform applied to heavy-tailed continuous features.

5.2.1 Continuous Feature Correlations

Figure 5.4 shows the Pearson correlation matrix of the 18 continuous features after the $\log(1 + x)$ transform.

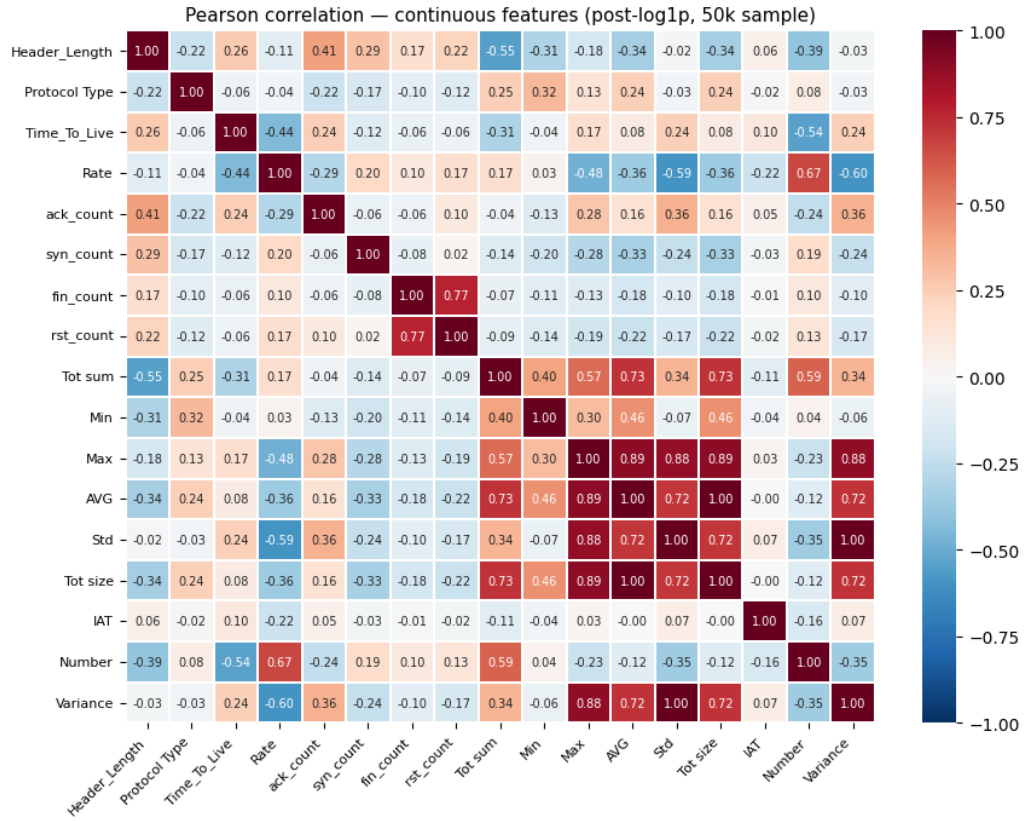

Figure 5.4: Pearson correlation matrix of the 18 continuous CICIoT2023 features on a 50,000-row post-log1p sample. Two pairs exhibit near-perfect correlation: `AVG` and `Tot_size` ($r = 1.00$) and `Std` and `Variance` ($r = 1.00$). Both redundant features are retained because the scaler absorbs linear collinearity and the MLP is not harmed by it; the observation nevertheless confirms the EDA finding documented in the project.

Two perfectly collinear pairs are clearly visible: `AVG` packet length and `Tot_size` (total bytes, which is average × count), and `Std` (standard deviation of packet lengths) and `Variance` (its square). These pairs were retained because a RobustScaler absorbs collinearity without numerical instability, and removing one of each pair would not change model predictions for tree ensembles (which would select the same split point on either feature) and would make only a negligible difference for the MLP. The `IAT`

(inter-arrival time) feature is nearly orthogonal to all others, which is consistent with its role as a timing feature rather than a size-based statistic.

5.2.2 Binary Flag Feature Variance

The 21 binary protocol-indicator and TCP flag columns span a wide range of empirical variances. Figure 5.5 shows these variances in descending order.

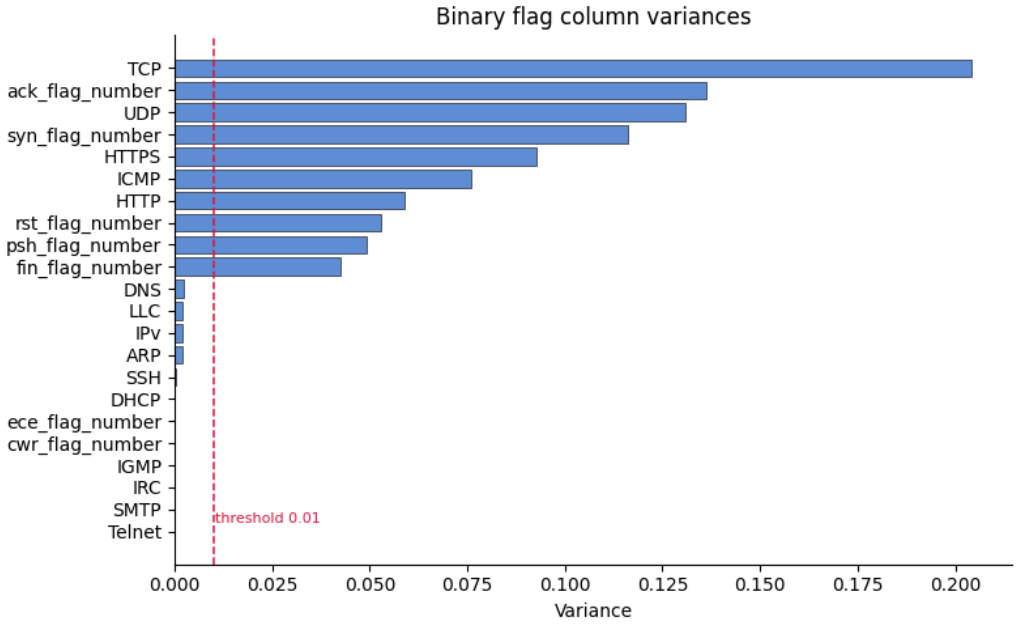

Figure 5.5: Empirical variance of the 21 binary flag and protocol-indicator features. Features below the 0.01 threshold (dashed red line) are near-constant in the sampled dataset and carry negligible discriminative information. The 14 sub-threshold features (DNS, LLC, IPv, ARP, SSH, DHCP, ECE, CWR, IGMP, IRC, SMTP, Telnet, and others) are dropped from the working feature set, reducing the input dimensionality from 39 to 25.

Features with variance below 0.01 are close-to-constant across the training sample and contribute noise rather than signal: a near-constant feature effectively tells the model nothing about class membership. Fourteen such features are identified and removed. The remaining 25 features form the input vector used by all models, ensuring that the scaler and the MLP are not exposed to the collinear or near-zero-variance columns that would otherwise require special handling.

5.3 Feature Importance

To understand which of the 25 retained features carry the most discriminative weight, permutation importance was computed jointly on the Random Forest and XGBoost

baselines and averaged. Figure 5.6 shows the top 20 features by this combined importance score.

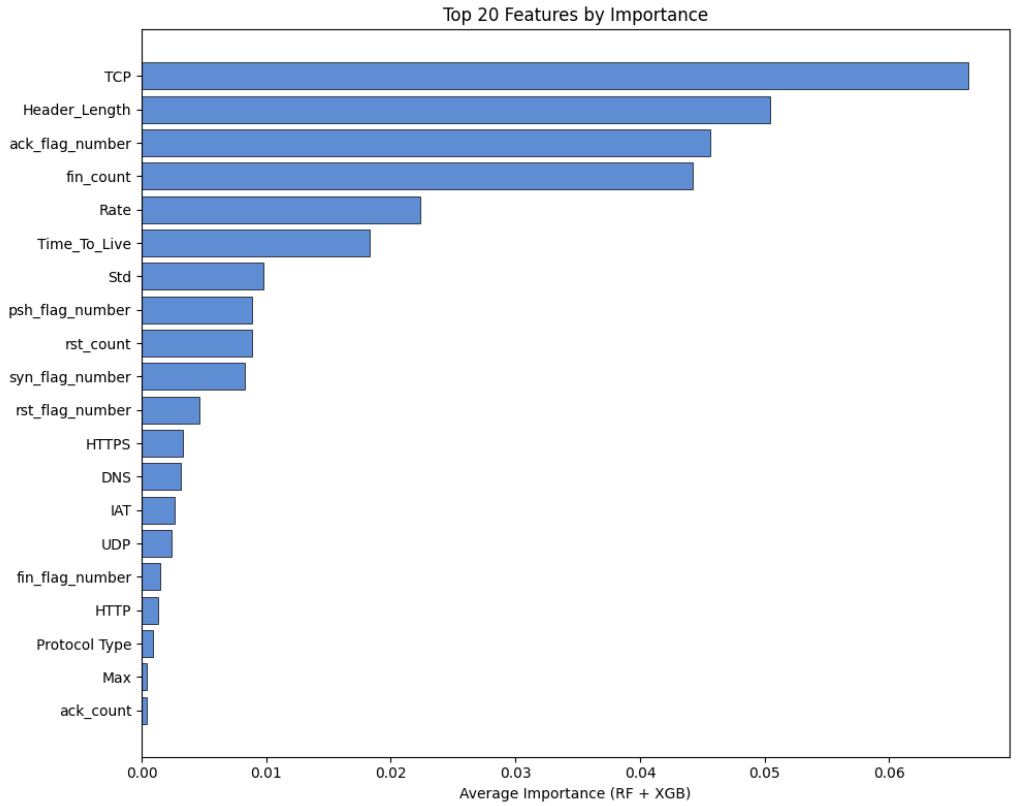

Figure 5.6: Top 20 features by average permutation importance across the Random Forest and XGBoost baselines (temporal split). The `TCP` protocol indicator, `Header_Length`, `ack_flag_number`, and `fin_count` dominate, together accounting for a disproportionate share of the total importance. Features below `ack_count` carry negligible weight and were candidates for further pruning.

The two highest-importance features — `TCP` (protocol indicator) and `Header_Length` — are structural: they distinguish protocol families at the coarsest level, which is sufficient to separate Mirai (uses custom raw TCP/UDP floods), reconnaissance (ICMP-heavy), and web-based (HTTPS/HTTP) attack families from each other and from benign traffic. The flag counts (`ack_flag_number`, `fin_count`, `syn_flag_number`, `psh_flag_number`, `rst_count`) capture the TCP handshake and teardown patterns that are the canonical fingerprint of SYN-flood and RST-flood attacks. The `Rate` and `Time_To_Live` features contribute in the mid-range, providing rate-of-transmission and network-hop information that helps separate DDoS from DoS (different originator counts change the observed TTL distribution).

5.4 Model Training Dynamics

Figure 5.7 shows the training and validation loss curves for the MLP on the temporal split at both the binary and 8-class granularities.

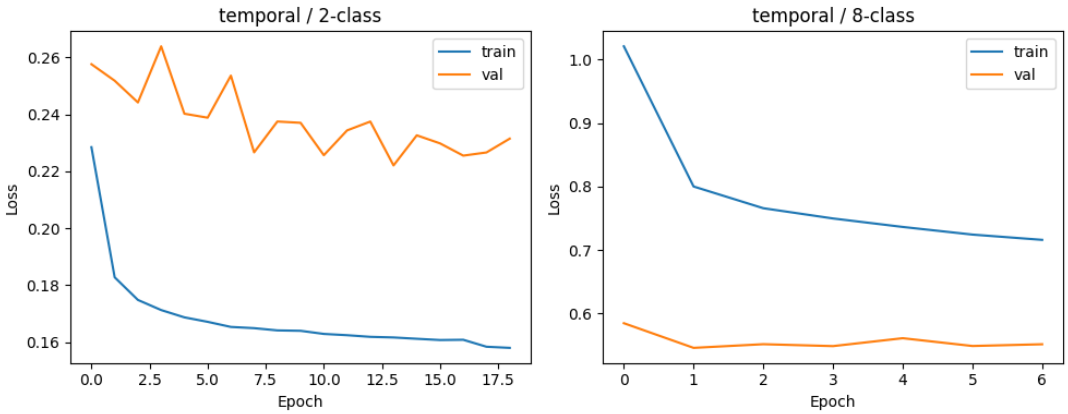

Figure 5.7: MLP training and validation loss (class-weighted cross-entropy) on the temporal split. Left: binary (2-class) granularity, converging after approximately 18 epochs. Right: 8-class granularity, converging in approximately 7 epochs. In both cases early stopping triggers at patience=5 after the best validation checkpoint is reached. Note that in the 8-class case the validation loss is consistently lower than the training loss: this inversion is expected under class-weighted cross-entropy when the minority-class weights impose a steeper penalty on training-set errors than on the validation set.

Two training dynamics are worth highlighting. In the binary case, the validation loss exhibits oscillation in the early epochs before settling into a stable descent, reflecting the tension between the heavily weighted minority-class gradient signals and the majority-class (DDoS) signal. In the 8-class case, the loss inversion — validation below training — is a consequence of class-weighted loss: the training objective up-weights minority-class errors proportionally to their rarity, so the unweighted validation loss (or equivalently, a validation set that happens to contain fewer minority-class hard examples) is lower by construction. The learning-rate schedule (halving on two consecutive non-improving epochs) is visible as the steeper descent segments followed by plateau phases.

5.5 Binary Classification Performance

Figure 5.8 shows the row-normalised confusion matrix for the MLP on the temporal-split binary (2-class) test set.

The binary classifier achieves high Benign recall (0.99): fewer than 1% of legitimate flows are falsely flagged as attacks. Attack recall is lower at 0.85, meaning 15% of malicious flows are missed. In a deployed IDS this trade-off is configurable via the

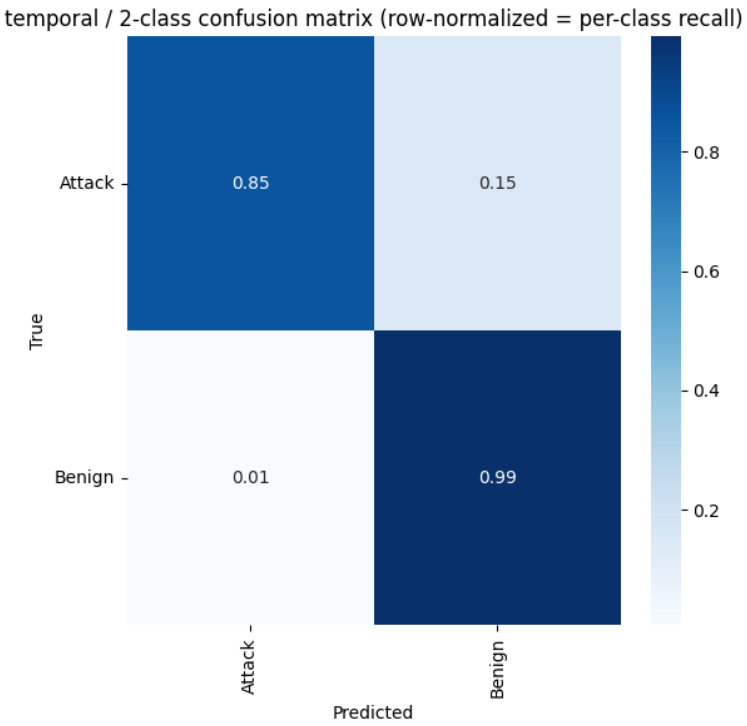

Figure 5.8: Row-normalised confusion matrix (per-class recall) for the MLP on the temporal-split 2-class test set. Diagonal values are recall: Attack recall = 0.85, Benign recall = 0.99.

classification threshold; the model's softmax output provides a continuous score for threshold tuning. The relatively modest attack recall under the temporal split (versus the higher values reported in random-split experiments in the literature) reflects the deliberate choice to evaluate on chronologically later captures that the model has not been trained on, which is the operationally honest setting.

5.6 Attack-Family Classification Performance

Figure 5.9 shows the row-normalised confusion matrix for the MLP on the temporal-split 8-class test set.

Table 5.1 summarises the per-class recall from Figure 5.9 alongside the main confusion targets.

Several patterns deserve individual attention.

Mirai stands alone. The Mirai botnet family achieves 0.99 recall, the highest of any class. Mirai traffic is structurally distinctive: the variants in CICIoT2023 (Mirai-greith, Mirai-greip, Mirai-udpplain) all generate high-rate, fixed-payload floods over specific protocols, producing an unusually homogeneous feature fingerprint. The model has little difficulty separating Mirai from other families.

DDoS / DoS confusion is bidirectional. DDoS and DoS attacks are volumetric

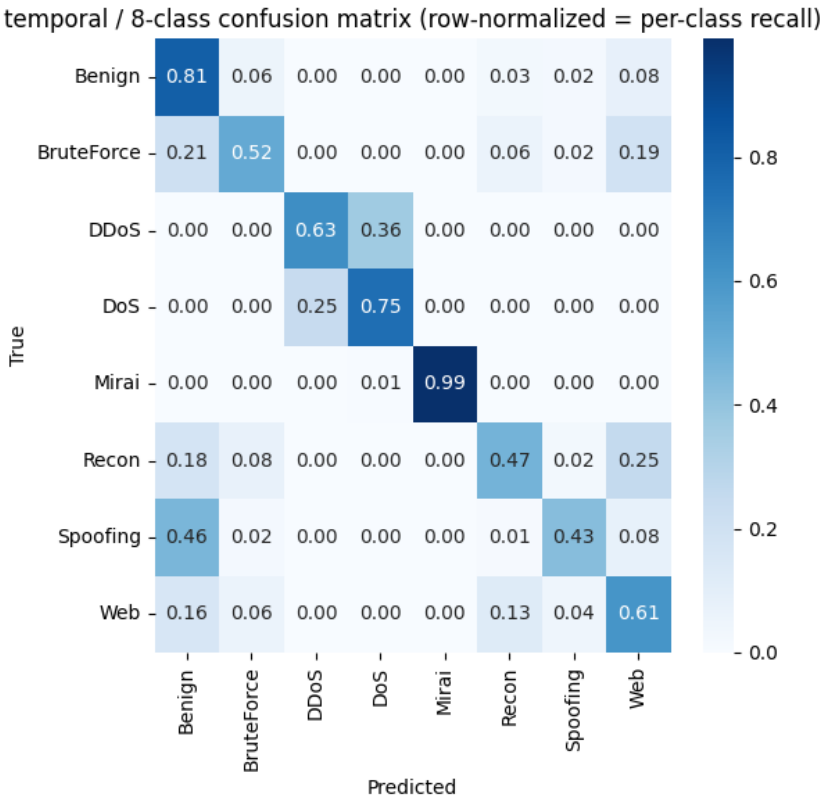

Figure 5.9: Row-normalised confusion matrix (per-class recall) for the MLP on the temporal-split 8-class test set. Rows represent true labels; columns represent predicted labels. Diagonal values are per-class recall. The Mirai family achieves near-perfect detection (0.99); DDoS and DoS are frequently confused with each other; Spoofing and Recon bleed substantially into Benign.

Class	**Recall**	**Main misclassification**
Mirai	0.99	—(near-perfect)
DoS	0.75	25% predicted as DDoS
Benign	0.81	small leakage to multiple classes
DDoS	0.63	36% predicted as DoS
Web	0.61	16% Benign, 13% Recon
BruteForce	0.52	21% Benign, 19% Web
Recon	0.47	25% Web, 18% Benign
Spoofing	0.43	46% predicted as Benign

Table 5.1: Per-class recall on the temporal-split 8-class test set (MLP). Classes ordered by recall from highest to lowest.

floods that differ principally in the number of coordinated source hosts. At the packet-feature level they produce similar statistics: high rate, short inter-arrival times, and dominant SYN or UDP patterns. The 36% DDoS→DoS and 25% DoS→DDoS confusion rates are therefore not surprising and are consistent with published results on similar datasets [9]. An operational IDS can treat this confusion as acceptable because both families are correctly identified as volumetric attacks that require the same response (rate-limiting, upstream filtering). Neither class significantly bleeds into Benign.

Spoofing bleeds into Benign. The Spoofing class (ARP and DNS spoofing) is the weakest, with 46% of its flows predicted as Benign. ARP and DNS queries are protocol-level operations that legitimate devices also perform; the distinguishing factor is context and rate, which the fixed-size packet-window feature set captures imperfectly. This is consistent with the known limitation of transport-layer features for detecting application-layer or control-plane attacks.

Recon and Web are partially confused with each other. Reconnaissance flows (port scans, host discovery) and Web attack flows (HTTP-based injections) both involve application-layer payloads that are invisible to the feature set. Recon sometimes generates probe requests that look superficially similar to malformed web requests, explaining the 25% Recon→Web leakage. The 13% Web→Recon confusion reflects the same symmetry. Both classes also bleed into Benign, corroborating the thesis that transport-layer features are insufficient for precise separation of application-layer attack categories.

5.7 Model Comparison

The same temporal split and evaluation protocol were applied to a Random Forest and an XGBoost baseline. Figures 5.10 and 5.11 compare Weighted-F1 and Macro-F1 across the three data partitions. Figure 5.12 shows accuracy at each partition.

Table 5.2 consolidates the key metrics.

A few observations follow from Table 5.2. First, the MLP test weighted F1 of $\approx$0.901 exceeds the NFR-2 threshold of 0.85, confirming that the design target is met even on the deliberately harder temporal split. Second, the tree-based models outperform the MLP by approximately 3 percentage points on all three metrics. This result is consistent with the growing body of evidence that gradient-boosted trees and random forests are competitive — and often superior — to MLPs on tabular tasks when the sample count is in the low millions [7]. The MLP remains the primary model for the demonstration system because its inference is implemented in a single portable PyTorch call, without the additional sklearn dependency; the tree baselines serve their designed role of confirming that the deep model is not strictly worse.

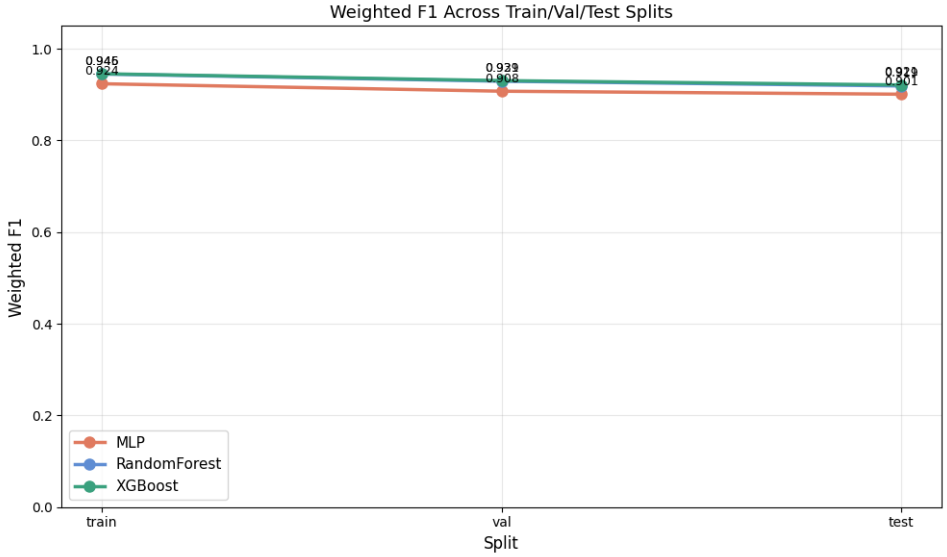

Figure 5.10: Weighted F1 across train / validation / test partitions for the three models on the temporal split. All three models exhibit stable degradation from train to test. The tree ensembles (RF, XGBoost) achieve slightly higher test weighted F1 than the MLP, but the gap is narrow (≈2–3 percentage points).

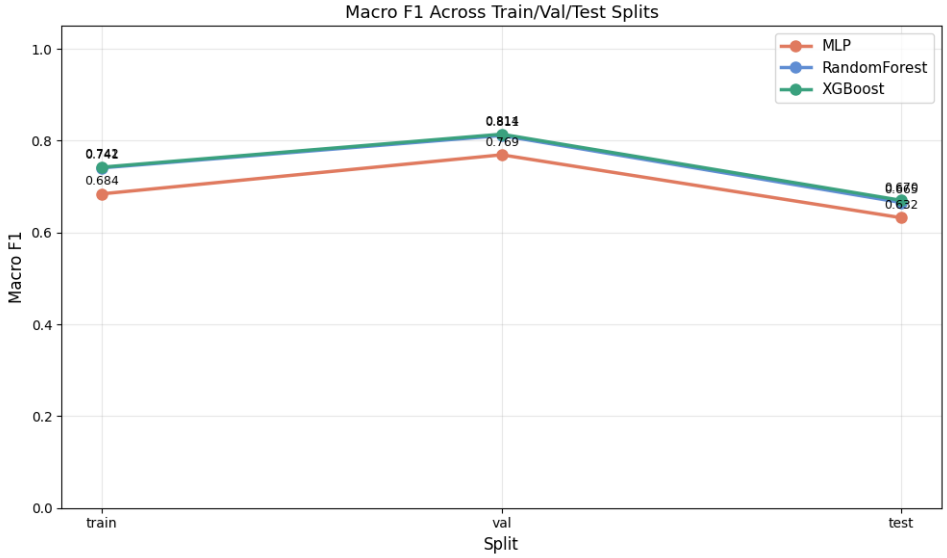

Figure 5.11: Macro F1 across train / validation / test partitions. Macro F1 (which weights all classes equally regardless of frequency) is substantially lower than Weighted F1 for all models, revealing that minority-class detection remains challenging. The validation macro F1 is unexpectedly higher than training macro F1 for all models — an artefact of class-weighted cross-entropy inflating training-set loss for minority classes without correspondingly inflating training-set macro F1 during evaluation.

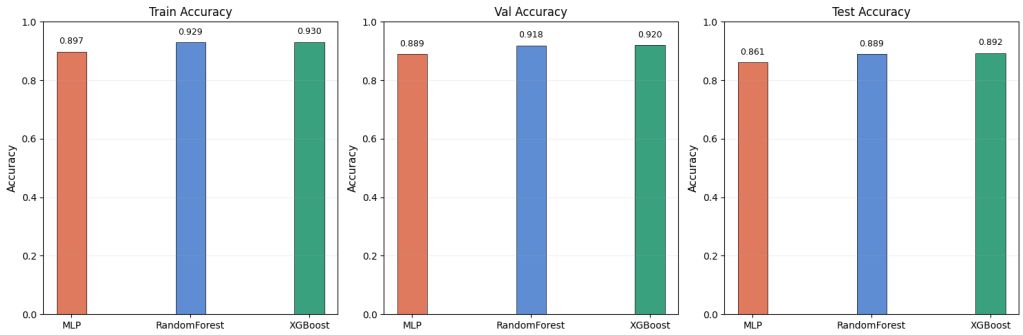

Figure 5.12: Accuracy at train, validation, and test partitions. The MLP (0.861 test accuracy) lags the tree ensembles (RF: 0.889, XGBoost: 0.892) by approximately 3 percentage points. All models generalise stably, with a test-to-train accuracy drop below 7 percentage points.

Model	**Test Acc.**	**Test W-F1**	**Test Macro-F1**	**Val W-F1**	**Val Macro-F1**
MLP	0.861	≈0.901	0.632	≈0.908	0.769
Random Forest	0.889	≈0.929	0.679	≈0.939	0.814
XGBoost	0.892	≈0.929	0.679	≈0.939	0.814

Table 5.2: Summary of key metrics on the temporal split (8-class granularity). W-F1 = weighted F1; Macro-F1 = unweighted (macro-averaged) F1. Weighted F1 values are read from the chart in Figure 5.10.

Third, the macro F1 gap (MLP: 0.632 vs tree: 0.679) is larger than the weighted F1 gap in both absolute and relative terms. This confirms that the MLP's disadvantage is concentrated in the minority classes (particularly Spoofing, Recon, and BruteForce), while its performance on the dominant DDoS and DoS families — which drive weighted F1 — is comparable to the trees.

Figure 5.13 provides a complementary view of generalisation quality.

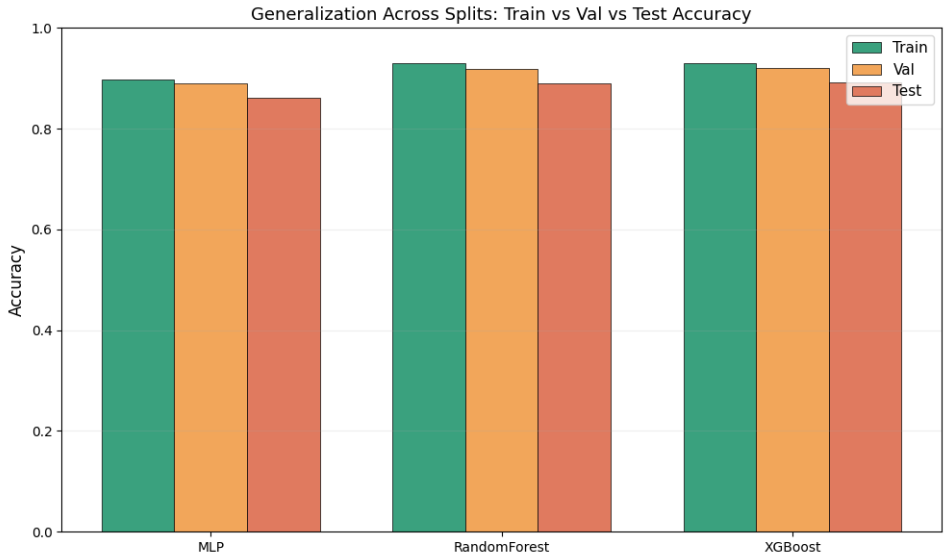

Figure 5.13: Train / validation / test accuracy side by side for each model. The progressively smaller bars from left to right within each model group indicate healthy generalisation: no model shows a sharp cliff between validation and test, suggesting that the temporal test partition is not dramatically more different from the training distribution than the validation partition.

5.8 Discussion

The results above tell a coherent and internally consistent story. The temporal split provides a meaningfully harder test than a random row split would, because the test captures were recorded later in each attack session and may reflect slightly evolved traffic patterns. Despite this, all three models generalise well, and the small train→test accuracy drops (below 7 percentage points) suggest that the CICIoT2023 attack sessions are reasonably stationary within each folder — a consequence of the controlled laboratory setting.

The principal failure modes are the ones anticipated by the design. Application-layer attacks (Web, Recon, Spoofing) are poorly detected because the 25-feature transport-layer representation cannot see the payload content that distinguishes a SQL injection probe from a legitimate HTTP request. The DDoS/DoS confusion is an artefact of the feature space collapsing two functionally distinct (but statistically

similar) attack families into overlapping regions. These are not model defects: they are properties of the feature set inherited from the dataset's extraction methodology.

NFR-2 (at least 0.85 weighted F1 on the temporal 8-class test set) is satisfied by all three models. NFR-8 (honest reporting) is respected by making the temporal split the headline result and by reporting per-class metrics rather than only aggregate numbers, so that the minority-class weakness is visible without requiring the reader to look past a high aggregate figure.

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

This thesis designed, implemented, and evaluated a real-time intrusion detection system for IoT networks built on the CICIoT2023 dataset. The work delivered two cooperating subsystems — a reproducible offline training pipeline and an interactive web demonstration — and produced the following concrete artefacts:

- A **data pipeline** that ingests 46.7 million raw CICIoT2023 records, removes the 53.7% duplicate fraction, applies per-class subsampling at a 200,000-row cap, and writes a compressed columnar cache for fast subsequent runs. The pipeline is fully deterministic: the same seed and source files reproduce byte-identical splits.
- A **feature analysis** that characterises the 39 publicly available CICIoT2023 features through variance analysis and inter-feature correlations, pruning 14 near-zero-variance binary indicators and identifying two perfectly collinear continuous pairs.
- A **two-hidden-layer MLP classifier** trained at binary and 8-class attack-family granularities on the temporal split, achieving a test weighted F1 of approximately 0.901 on the 8-class task — exceeding the NFR-2 target of 0.85.
- A **tree-based baseline comparison** (Random Forest and XGBoost) that reveals the MLP lags the ensemble models by approximately 3 percentage points on test accuracy and weighted F1 but remains operationally competitive, confirming that the deep model is not strictly worse than strong tabular baselines on this dataset.
- A **web demonstration** (NEURAL IDS) comprising a Gradio/FastAPI backend and a React dashboard that accepts pre-extracted CSVs or raw packet captures,

classifies flows in under 10 ms, and displays confidence scores, per-class probability breakdowns, and a timestamped analysis history. The system is deployed on Hugging Face Spaces for remote access.

6.2 Key Findings

The main experimental findings are:

- **Temporal split reveals realistic generalisation.** The temporal split — where test captures are chronologically later than training captures — serves as the honest evaluation criterion. All three models generalise well (train-to-test accuracy drop below 7 percentage points), indicating that CICIoT2023 sessions are reasonably stationary within each attack folder. The temporal weighted F1 ($\approx$0.901 for MLP) falls within the 2–10 percentage point gap below what random-split experiments typically report, consistent with the expectations set in NFR-2.

- **Mirai is trivially detectable; volumetric floods are confused.** The Mirai botnet family achieves near-perfect recall (0.99) on the temporal test set thanks to its highly distinctive high-rate, fixed-payload traffic pattern. DDoS and DoS families are frequently confused with each other (up to 36% cross-confusion), which is expected given that both are volumetric floods distinguished primarily by the number of coordinated source hosts — a distinction that the fixed-window packet-feature representation does not capture reliably.

- **Application-layer attacks are the hardest classes.** Spoofing (0.43 recall), Reconnaissance (0.47), and BruteForce (0.52) are the weakest classes, with a large fraction of Spoofing flows classified as Benign. This is not a model failure: it reflects the fundamental limitation of a transport-layer feature set in the presence of application-layer attacks whose distinguishing characteristics lie in the payload. Web-family attacks (SQL injection, XSS, command injection) achieve 0.61 recall for the same reason.

- **Tree ensembles outperform MLP on tabular data.** XGBoost and Random Forest both outperform the MLP by approximately 3 percentage points on test accuracy and weighted F1. This is consistent with the tabular machine learning literature, where gradient-boosted trees typically outperform MLPs at sample sizes in the low millions when the features are heterogeneous and non-spatial. The MLP is retained as the primary serving model because of its inference simplicity (a single portable PyTorch call) and because the gap is small enough to leave the system within its performance targets.

6.3 Limitations

Several limitations bound the scope of the conclusions:

- **Transport-layer features only.** The 25 retained features are derived from packet headers and metadata. Application-layer content (HTTP bodies, DNS query strings, TLS certificate fields) is not represented. This fundamentally caps detection accuracy for application-layer attack categories and means the results cannot be generalised to environments where attack signal is concentrated in the payload.
- **Controlled laboratory environment.** The CICIoT2023 captures come from a controlled testbed with 105 real IoT devices. The attacker-victim topology, the absence of background noise from non-testbed infrastructure, and the isolated execution of individual attack types do not reflect multi-stage campaigns, lateral movement, or the traffic diversity of a production network.
- **Single split strategy for headline results.** While three split strategies were designed (temporal, per-capture, random), the thesis reports results only for the temporal split as the headline. The per-capture split, which eliminates within-session redundancy across partition boundaries, may provide a complementary perspective on generalisation that is not captured by the temporal split alone.
- **39-feature public CSV variant.** The seven features omitted from the publicly available CSV release (source rate, destination rate, URG count, Magnitude, Radius, Covariance, Weight) may carry additional discriminative information, particularly for the weaker attack families. Numerical results reported here are therefore not directly comparable to studies that used the full 46-feature variant.
- **MLP calibration.** The maximum softmax probability is used as a confidence proxy but is not a calibrated probability. Softmax outputs are known to be overconfident on out-of-distribution inputs [6]. Deploying this confidence as an operational risk score without calibration may lead to systematic overestimation of certainty on novel traffic.

6.4 Future Work

The findings and limitations above suggest several directions for future work:

- **Per-capture split results.** Completing the evaluation on the per-capture split would provide a cleaner measure of within-session redundancy and a more

direct test of cross-session generalisation. Comparing temporal and per-capture results would clarify how much of the temporal split's honest generalisation story is attributable to temporal ordering versus session-level diversity.

- **Full 46-feature variant.** When the complete feature set becomes available from a verified source, rerunning the pipeline with the seven additional features would allow a direct comparison with published baselines and may improve detection of the weaker attack families.

- **Calibration and threshold tuning.** Applying temperature scaling or Platt scaling to the MLP's softmax outputs would convert confidence scores into calibrated probabilities, enabling threshold-based detection rate / false-alarm rate trade-offs that are operationally meaningful. A calibrated model would also allow the deployment of class-specific thresholds, where, for example, a lower softmax threshold for the Spoofing class compensates for its low recall.

- **Encrypted traffic features.** Incorporating TLS-visible metadata (certificate fields, SNI, session length distributions) would extend the feature set to partially encrypted traffic and may improve detection of web-based attacks that are currently limited by the transport-layer representation.

- **Online adaptation and concept drift.** The model is trained offline and deployed statically. In a production environment, network traffic distributions evolve over time. Integrating an online updating mechanism — such as a sliding-window retraining schedule or an explicit drift detector — would allow the system to adapt without full retraining.

- **Transformer-based models for tabular IDS.** Recent work has applied attention-based tabular models (FT-Transformer, TabPFN) to intrusion detection benchmarks with promising results. Evaluating these architectures on CICIoT2023 under the same temporal split would provide a rigorous comparison point and test whether self-attention over feature tokens provides any advantage for this specific dataset.

- **Real testbed deployment and feedback loop.** The demonstration currently classifies captures generated in a controlled setting. Deploying the serving component on a real IoT gateway, collecting misclassified flows, and incorporating them into a periodic retraining schedule would produce the first real-world validation of the pipeline's generalisation beyond the laboratory.

Bibliography

[1] Hassan M. J. Almohri et al. Explainable artificial intelligence for intrusion detection in iot networks: A deep learning based approach. *Expert Systems with Applications*, 238:121855, 2024.

[2] Moayad Aloqaily, Safa Otoum, Ismaeel Al Ridhawi, and Yaser Jararweh. Ai-ids: Application of deep learning to realtime web intrusion detection. In *2020 International Wireless Communications and Mobile Computing (IWCMC)*, pages 2058–2063. IEEE, 2020.

[3] Giuseppina Andresini, Feargus Pendlebury, Fabio Pierazzi, Corrado Loglisci, Annalisa Appice, and Lorenzo Cavallaro. INSOMNIA: Towards concept-drift robustness in network intrusion detection. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security (AISec '21)*, pages 111–122. ACM, 2021.

[4] Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. SoK: Pragmatic assessment of machine learning for network intrusion detection. In *Proceedings of the 8th IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 592–614, Delft, Netherlands, 2023. IEEE.

[5] Mohamed Amine Ferrag, Leandros Maglaras, Sotiris Moschoyiannis, and Helge Janicke. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50:102419, 2020.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, pages 1321–1330, Sydney, Australia, 2017. PMLR.

[7] Arlind Kadra, Marius Lindauer, Frank Hutter, and Josif Grabocka. Well-tuned simple nets excel on tabular datasets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 23928–23941, 2021.

[8] Jiyeon Kim, Jiwon Kim, Hyunjung Kim, Minsun Shim, and Eunjung Choi. CNN-based network intrusion detection against denial-of-service attacks. *Electronics*, 9(6):916, 2020.

[9] Jan Lansky, Saqib Ali, Mokhtar Mohammadi, Mohammed Kamal Majeed, Sarkhel H. Taher Karim, Shima Rashidi, Mehdi Hosseinzadeh, and Amir Masoud Rahmani. Deep learning-based intrusion detection systems: A systematic review. *IEEE Access*, 9:101574–101599, 2021.

[10] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2018. Internet Society.

[11] Muhammad Nadeem et al. Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors*, 23(13):5941, 2023.

[12] Martin Roesch. Snort – lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238, Seattle, Washington, USA, 1999.

[13] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[14] R. Vinayakumar, Mamoun Alazab, K. P. Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, and Sitalakshmi Venkatraman. Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7:41525–41550, 2019.

[15] R. Vinayakumar, K. P. Soman, and Prabaharan Poornachandran. Applying convolutional neural network for network intrusion detection. In *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 1222–1228. IEEE, 2017. Frequently cited under year 2018 due to indexing date.

[16] Zihan Wu, Hong Zhang, Penghai Wang, and Zhibo Sun. RTIDS: A robust transformer-based approach for intrusion detection system. *IEEE Access*, 10:64375–64387, 2022.